BitBull

Short-Dated BTC Binary Options Exchange
System Architecture, Quantitative Research & Engineering Record

Quantitative Research Internship · GameBull / Baazi Games

Crypto Vertical — House-Made-Market Prediction Exchange

Prepared by: Sidharth Jain

Date: 31 July 2026

Audience: Lead Engineering — no prior context assumed

0 · Executive Summary

BitBull is a working prediction-market exchange for **5-minute BTC binary options** ("Will BTC be ≥ $X at time T?"), built to answer one commercial question: **can the house profitably make markets on short-dated crypto binaries, and if so, what machinery does it need?**

Unlike a conventional order-book exchange that only collects fees, here **the house is the counterparty to every trade**. That means it carries real directional inventory risk. The work therefore splits into three problems: price it correctly, quote it profitably, and hedge what's left over.

> **Headline results**
>
> - **Pricing.** Constant-volatility Black–Scholes mispriced live short-dated binaries by **35.9 percentage points** (mean absolute error) against real Kalshi order-book data. A √τ-scaled empirically-calibrated curve cut that to **5.0 pp — a ~7× improvement**. Now live in production.
> - **Profitability.** The dominant loss driver was identified as **short-gamma pin risk**, not spread mis-setting. A gamma-aware quote-widening term took profitable 5-minute windows from **78%** → **95%** and cut P&L volatility **36%**; layered inventory/risk controls reached **97% / –42%**.
> - **Hedging.** A read-only delta-hedging sidecar maps house LMSR inventory to perpetual-futures hedges, removing **~35% of P&L dispersion** and turning worst-case windows from a loss into a gain.
> - **The honest limit.** Perps **provably cannot** hedge a binary's terminal gamma. This is a mathematical fact, not an implementation gap — documented in §8 along with two attempted remedies, one of which returned a clean **NO-GO**.
> - **The economic limit, found late.** At *full* delta size the hedge requires **300× more perp notional than the position can possibly lose**, costing **$429 per window** in taker fees against **$54** of spread revenue — a **24¢ break-even spread** where 3–6¢ is observed. Full delta hedging is therefore uneconomic, and the correct posture is a **partial, maker-executed hedge** (~0.30 captures two-thirds of the benefit at under a third of the cost). See §8.7 — this materially revises the priority order in §11.1.

This document is written so that a reader with **no prior exposure** to the platform can follow it end to end: what was built, why each decision was made, what the mathematics says, and — importantly — **where the theory does not go in our favour**.

Contents

1 · The Assignment and Its Constraints

1.1 Business context

GameBull (Baazi Games) operates a prediction-market platform whose existing product line is sports-focused. The mandate was to determine the viability of a **crypto vertical** — short-dated BTC binaries — and build the pricing, risk and hedging machinery required to run it.

The critical structural difference from a traditional exchange:

```
Order-book exchange: revenue = fees, risk ≈ 0
House-made market: revenue = spread, risk = full directional inventory
```

1.2 The hard constraint

> **Read-only production access.** Production repositories could be **read but never modified**. Every component built had to interact with the company's real services exactly as an external client would — through existing APIs, queues and data stores. No pull requests, no schema changes, no patches to their services.

This constraint drove much of the architecture. It is the reason a full local mirror of the production stack exists (§2.4), and the reason the hedging service is a strictly read-only sidecar (§8.2) rather than an in-process change to the matching engine.

1.3 What "a 5-minute binary" actually is

A binary (digital) option paying **$1 if BTC ≥ strike at expiry, $0 otherwise**. Its fair value is the risk-neutral probability of finishing in the money. Two properties make it uniquely hard to make markets in:

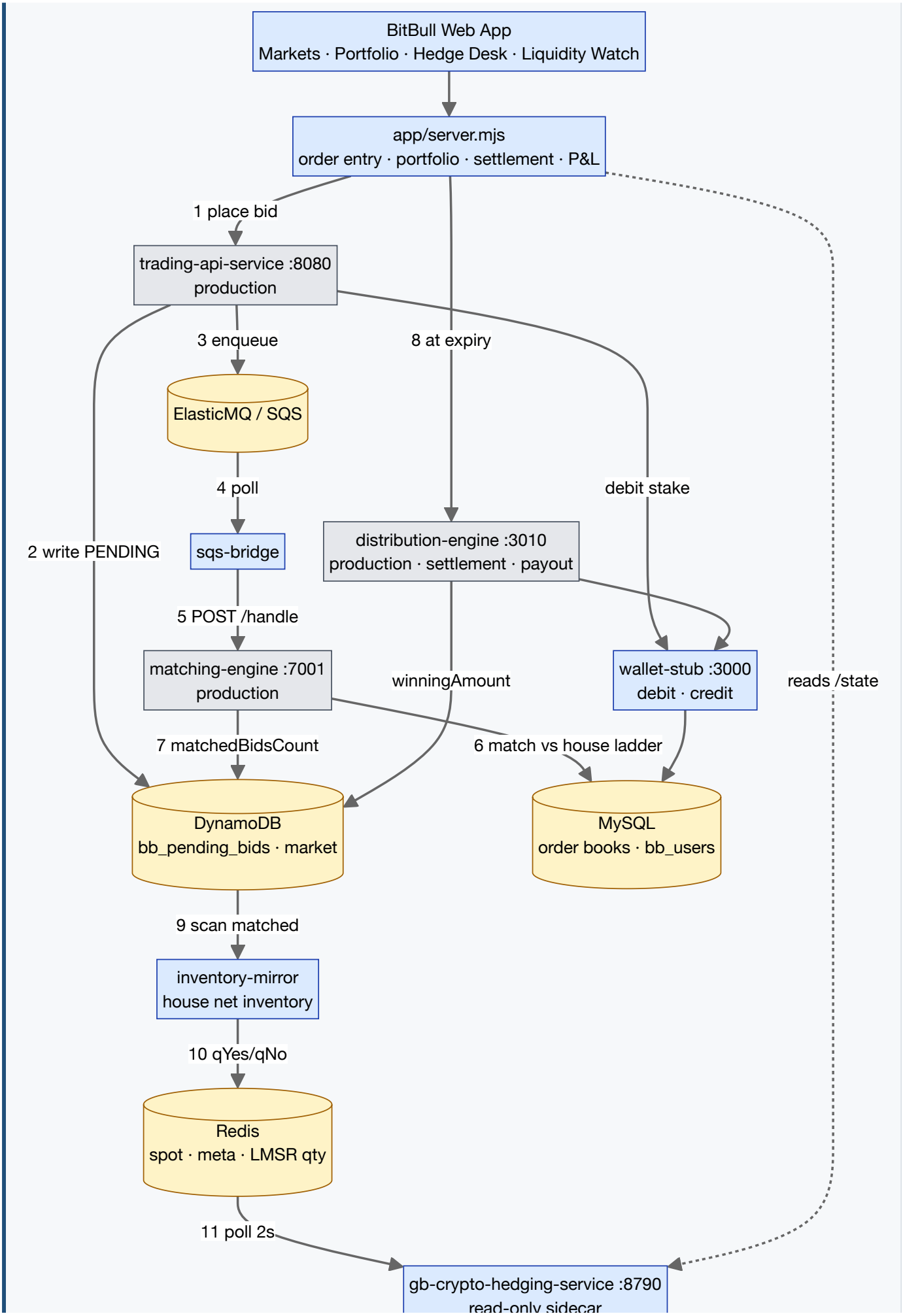
- **Discontinuous payoff.** The payoff is a step function. Value jumps from ~0 to ~1 across an infinitesimal move in spot at expiry.
- **Diverging sensitivity.** The rate of change of value with respect to spot (delta) **tends to infinity** as time to expiry $\tau \to 0$ near the strike. This is the source of essentially every difficulty in this document.

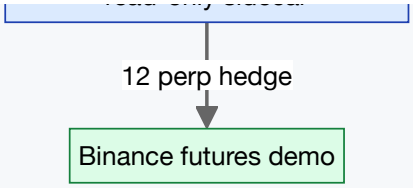
2 · System Architecture — Full Topology

2.1 The complete picture

The request path below is the spine of the platform: a user order travels through GameBull's **real, unmodified** production services end to end, with only the queue bridge substituted so their stack can run locally.

Legend: blue = built for this project · grey = GameBull production, unmodified · green = external venue/feed · amber = data store


BitBull Web App
Markets · Portfolio · Hedge Desk · Liquidity Watch
app/server.mjs
order entry · portfolio · settlement · P&L
1 place bid
trading-api-service :8080
production
3 enqueue
8 at expiry
ElasticMQ / SQS
4 poll
debit stake
2 write PENDING
sqs-bridge
distribution-engine :3010
production · settlement · payout
5 POST /handle
matching-engine :7001
production
winningAmount
wallet-stub :3000
debit · credit
reads /state
7 matchedBidsCount
6 match vs house ladder
DynamoDB
bb_pending_bids · market
MySQL
order books · bb_users
9 scan matched
inventory-mirror
house net inventory
10 qYes/qNo
Redis
spot · meta · LMSR qty
11 poll 2s
gb-crypto-hedging-service :8790
read-only sidecar



The one seam that is not their production code. Steps 4–5 substitute `sqs-bridge` for AWS SQS delivery, because their service derives its request host from the queue URL (§2.4). Everything else on this path — order validation, matching, settlement, payout — is GameBull's **real, unmodified** service code.

2.2 Data-flow map — who writes what, and who reads it

The diagram above shows the *request* path with numbered steps. This table shows **state**: every store the platform depends on, its exact key or table name, what it holds, and — critically — **who writes it and who reads it**. A many-to-many dependency map of this shape is stated more precisely as a table than as a graph.

Store / key	Contents	Written by	Read by
`Redis` `CRYPTO_SPOT_BTCUSDT`	Latest BTC spot, JSON with timestamp	oracle-feed (250ms throttle)	app/server.mjs, mmp-pricing, liquidity-watcher
`Redis` `MMP_MARKET_META_{id}`	strike, expiryTs, symbol, feedId	market-generator	app/server.mjs, **hedging-service**, liquidity-watcher
`Redis` `MMP_LMSR_QUANTITY_YES|NO_{id}`	House inventory, sign-swapped so `qYes = houseNo` (§8.2)	**inventory-mirror**	**hedging-service** — its only inventory source
`Redis` `predictor_active_markets`	Set of live market IDs	market-generator, settlement	app/server.mjs, hedging-service, mmp-pricing, liquidity-watcher
`Redis` `liquidity_watch_state`	Live gap state, 30s TTL so a dead watcher reads as stale rather than silent	liquidity-watcher	app/server.mjs → Liquidity Watch tab
`MySQL` `bb_available_bids_{yes,no}_{marketId}`	Resting order book, one table per market per side	matching-engine, mmp-pricing	app/server.mjs, mmp-pricing, liquidity-watcher
`MySQL` `bb_users`	Wallet balances in **points** (100 pts = $1)	wallet-stub	app/server.mjs
`DynamoDB` `bb_pending_bids`	Every bid + `matchedBidsCount` + `winningAmount`	trading-api, matching-engine, distribution-engine	app/server.mjs, **inventory-mirror**
`DynamoDB` `market`	Market record, resolved `answer` after settlement	market-generator, app/server.mjs	app/server.mjs, trading-api

Note on scale. Both `bb_pending_bids` readers perform full-table scans. This is the correctness-critical path where the pagination defect of §10.5 lived, and it remains *O(table)* per call — a per-market Query on the `mkId` GSI is the outstanding scale fix.

Two independent spot feeds — deliberate, and worth knowing. The exchange gets spot via `oracle-feed` → Binance WebSocket → Redis `CRYPTO_SPOT_BTCUSDT` (throttled to one write per 250ms). The hedging sidecar does **not** read that key — it maintains its **own** self-reconnecting Binance WebSocket feed with a staleness watchdog.

This is intentional: the hedger must not inherit a stale or wedged exchange feed when deciding to trade real money, and it must remain functional if the exchange's Redis is unavailable. The cost is two WebSocket connections and the possibility of momentary disagreement between the price the exchange quotes on and the price the hedger sizes against.

Reading the inventory path (steps 9–11). The hedging sidecar never reads DynamoDB or MySQL. `inventory-mirror` scans `bb_pending_bids`, sums the **house's** matched fills per market, and republishes them as the two LMSR quantity keys. The sidecar consumes only those keys plus market metadata. This keeps the sidecar's contract identical locally and in production, and is why it can be truthfully described as read-only against GameBull's stores.

Supporting services — the remaining processes, none of which sit on the order path:

Process	Role
`oracle-feed`	Binance WebSocket → Redis spot key, throttled to 250ms writes.
`wallet-stub` :3000	The one component still stubbed rather than real: fetch-wallet, debit/credit, batch-process against `bb_users`. Accounting is genuine.
`secrets-stub`	Stands in for AWS Secrets Manager. Required because the distribution engine hardcodes `port 3000` **and** `wallet.base_url = localhost:3000` — locally self-contradictory, so it would bind a port and then call itself for payouts. Overriding those two values via the secret is the only external way to resolve it.
`market-generator`	Invoked on demand by the lifecycle loop, not a daemon — creates the next rolling market when none is live.

2.3 Component responsibilities

Component	Responsibility
`app/server.mjs`	User-facing API. Order entry (market/limit), portfolio, settlement orchestration, market-maker P&L and calibration accounting.
`drivers/lib/pricing.mjs`	**Fair value.** Empirical σ√τ-scaled probability curve (§4) plus Black–Scholes digital reference.
`drivers/lib/quoting.mjs`	**Spread construction.** Avellaneda–Stoikov reservation pricing plus the gamma/inventory/flow widening terms (§6).
`drivers/lib/risk.mjs`	Inventory limits and quote sizing.
`drivers/mmp-pricing`	Persistent quoting worker. Cancels stale ladders and reposts two-sided quotes every ~2s.
`drivers/market-generator`	Creates rolling at-the-money markets; tenor configurable (5m / 15m / 1h).
`drivers/sqs-bridge`	Drains SQS queues into the real matching engine — the seam that lets their service run locally.
`drivers/inventory-mirror`	Publishes house net inventory into the Redis keys the hedging service consumes.
`drivers/liquidity-watcher`	Diagnostic. Polls every book at 400ms, records any genuinely-empty side (§10.2).
`drivers/oracle-feed`	Binance WebSocket → Redis `CRYPTO_SPOT_BTCUSDT`, throttled to 250ms writes. The exchange's price source.
`drivers/wallet-stub`	The only component still stubbed rather than real. Serves fetch-wallet, debit/credit and batch-process against `bb_users` — the accounting itself is genuine.
`drivers/secrets-stub`	AWS Secrets Manager stand-in; resolves a hardcoded port/URL collision in the distribution engine (see §2.4).
`gb-crypto-hedging-service`	Read-only perp-hedging sidecar (§8.2). Separate repo, deployable, containerised.

2.4 Why a full local mirror of production exists

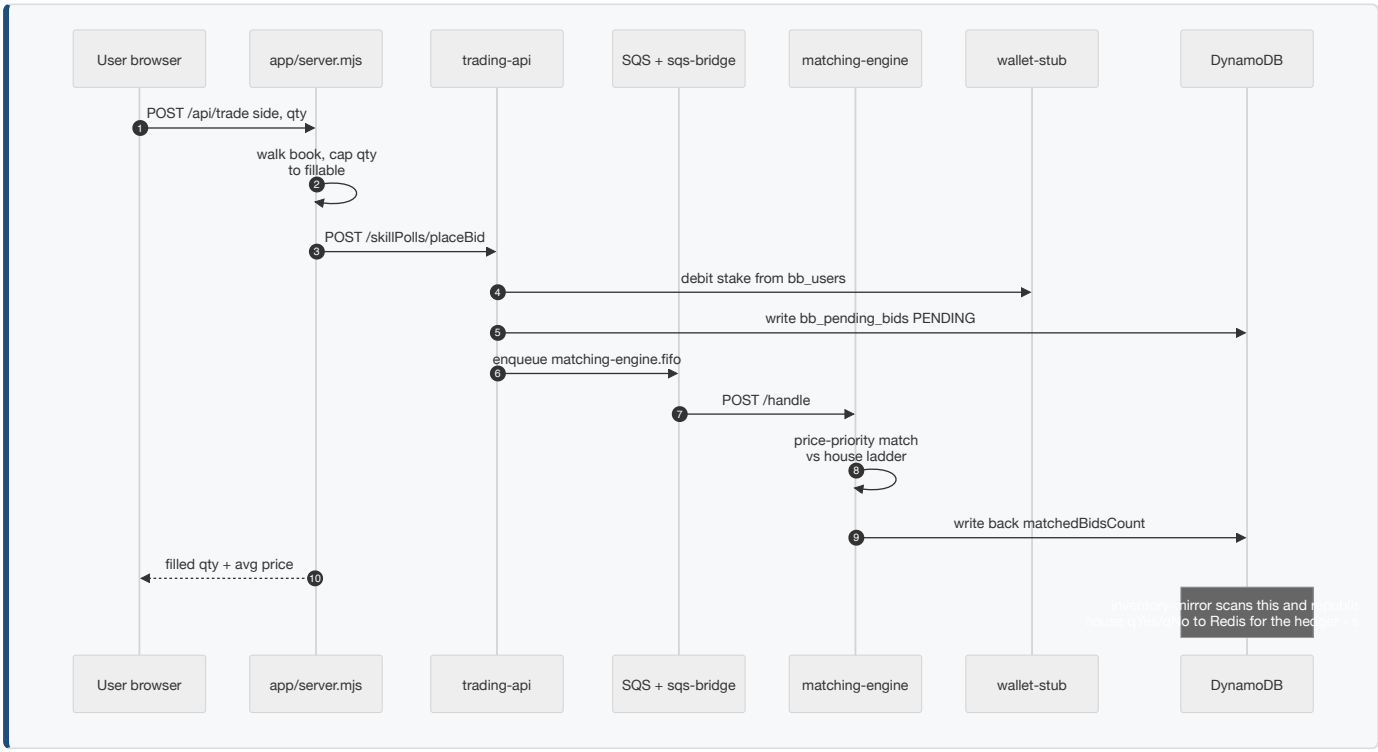
To test against reality without touching production, GameBull's **real, unmodified** services were stood up locally against a Docker stack. Three integration seams had to be solved entirely from the outside:

Seam	Problem	External solution
Auth	Service requires MMP auth plus an IP allowlist.	Existing env flag `SKIP_MMP_AUTH` reduces the check to an API key header — no code change.
SQS	aws-sdk v2 derives the request host from the `QueueUrl` parameter and **ignores the configured client endpoint**, so their service kept calling real AWS and returning 403.	A preload shim rewrites `QueueUrl` to localhost. Subtlety: v2 attaches operations **lazily**, so prototype methods are undefined at preload time — each SQS **instance's** methods had to be wrapped instead.
Market shape	Their controller reads market records from DynamoDB (not MySQL) and destructures several nested fields; missing ones crash the request.	The market generator emits fully conforming records.

Result: a real user order flows through their actual trading-api → DynamoDB → SQS → their actual matching engine → their actual distribution engine for settlement and payout. This is a genuine integration environment, not a simulation of one.

3 · The Life of a Trade

This sequence is the single most useful diagram for understanding the platform. It is the path exercised by the live verification in §11.



3.1 Settlement

At expiry the lifecycle loop settles the market: the distribution engine assigns and distributes winnings, then `settle()` credits winner wallets and books house P&L. A settled binary is worth **exactly 100¢ or 0¢** — an earlier defect marked expired positions off a live model value with τ pinned at 1s, so P&L drifted forever and positions never visibly resolved.

> **Units — a real bug worth stating.** `bb_users.unused_amount` and `winningAmount` are denominated in **points** (100 points = $1), while P&L display is in dollars. Passing the dollar figure to the wallet credited **100× too little** — a 100-contract win paid $1 instead of $100. Fixed; the conversion is now explicit and commented at the call site.

3.2 The three price layers, and how positions are marked

Three distinct numbers describe the same market at the same instant, and conflating them is the most common source of confusion when reading the app:

Layer	Value	What it is	Tradeable?
1 — fair value	`fairYes`	The **model's** probability (§4). Quoting input, analytics, risk gating.	**No.** Never displayed as a price.
2 — tradeable	`bestBid` / `bestAsk`	The touch of the resting book.	**Yes** — what users buy and sell at.
3 — market price	`marketPrice`	Book **midpoint**, falling back to last trade.	No — a reference number.

Layers 1 and 3 legitimately differ — observed live at fair **62.0¢** against a mid of **63.1¢** on a 60.1/66.1 book. The gap comes from inventory skew, tick rounding, and asymmetry between the two ladders.

Why `avg` and `mark` differ in a portfolio

`avg` is what the user **paid** — the ask. `mark` is the current value of the position. Previously `mark` was the **model fair value**, which meant buying at the ask and marking to the model showed an instant unrealised loss of the entire spread the moment a trade filled — measured against a number the user could never have transacted at. Live example: fair 62¢ versus ask 66.1¢, i.e. **–4.1¢/contract before anything happened**. It also disagreed with the market-detail chart, which already plots the mid.

> **What the venues actually do** — neither marks to a model. **Polymarket** displays the bid/ask **midpoint** as the probability, falling back to the **last traded price** when the spread exceeds $0.10. **Kalshi** charts the **last traded** Yes price.

Changed to mark-to-mid, with fair value retained only as a fallback for a market with no resting book (brand-new, or one side dark under directional lockout) so a position is never unpriced. Verified live: avg 51.6¢, mid 50.05¢, unrealised −0.09 = 6 × (50.05 − 51.6)/100 — the arithmetic confirms mid rather than fair, which would have given −0.10.

> **Action item — switch to mark-to-BID when selling is implemented.** Mid is correct *only because* positions are currently hold-to-settlement: the app exposes BUY YES / BUY NO with no sell or close path, so mid is the fair reporting value of something that cannot be liquidated. Once a position *is* liquidatable, its value is what you would receive on exit — the **bid** (YES at `bestBid`, NO at `100 − bestAsk`). Kalshi states this directly: "*the midpoint may be 58¢ but the bid (where you actually sell into) might be 57¢. Always look at the actual bid price, not the midpoint, when computing your realized exit value.*" Marking a liquidatable position at mid overstates portfolio value by half the spread on *every* open position, and makes realised P&L drop by that amount the instant a user closes. Written up in `docs/pricing-and-quoting.md`.

4 · Pricing Engine — From Black–Scholes to an Empirical Curve

4.1 The starting point

Fair value for a digital under Black–Scholes with zero rates:

$$p = N(d), \quad d = [\ln(S/K) - \tfrac{1}{2}\sigma^2\tau] / (\sigma\sqrt{\tau})$$

with delta and gamma

$$\partial p/\partial S = \varphi(d) / (S\cdot\sigma\sqrt{\tau}) \qquad \partial^2 p/\partial S^2 = -\varphi(d)\cdot d_1 / (S^2\sigma^2\tau)$$

Both are implemented in `core/digital.ts`. Note the **1/τ** in gamma — this term is the entire subject of §6.

4.2 Why Black-Scholes fails here — the decisive test

Black–Scholes assumes **constant volatility**: one σ describes the underlying for the whole life of the option. For a 3-month equity option that is a workable idealisation. For a **5-minute** BTC binary it is not, and the failure is measurable rather than a matter of opinion.

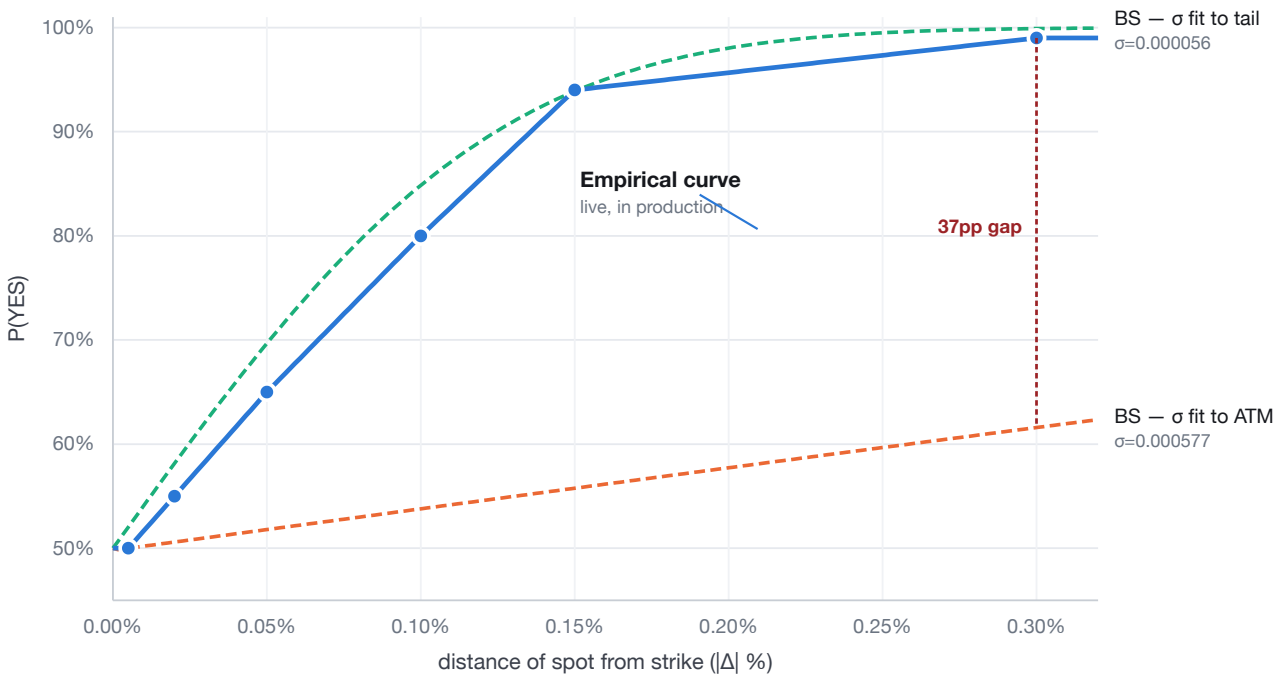
The clean test is to ask: **what σ would Black–Scholes need in order to reproduce each observed market price?** If BS were the right model, every point would imply roughly the *same* σ. Inverting the digital formula at each calibration point (τ = 300s) gives:

Distance from strike	Observed P(YES)	σ per second that BS would require
0.005%	50%	**0.000577** ← needs HIGH vol: "this move is just noise"
0.02%	55%	0.000091
0.05%	65%	0.000075
0.10%	80%	0.000069
0.15%	94%	**0.000056** ← needs LOW vol: "this move is decisive"
0.30%	99%	0.000074

> **The implied σ varies by 10.4× across the same curve, at the same instant, on the same asset.** That is not a calibration difficulty — it is the model being structurally wrong. A constant-σ diffusion **cannot** be simultaneously flat near the money and steep in the tails, which is exactly the shape short-dated crypto binaries actually trade at.

What that looks like

Pick either σ and Black–Scholes fails at the opposite end of the curve:



At 0.30% from strike	P(YES)	Error vs market
Empirical curve	**99%**	—
BS, σ fitted to the ATM point	62%	**37pp too low** — wildly under-reactive
BS, σ fitted to the tail	100%	close here…
At 0.02% from strike	**P(YES)**	**Error vs market**
Empirical curve	**55%**	—
BS, σ fitted to the ATM point	51%	4pp too low
BS, σ fitted to the tail	58%	3pp too high — and it hit 100% at 0.30%, over-confident at the pin

Fitting σ to the middle splits the difference and is wrong at *both* ends. This is the concrete mechanism behind the headline **35.9pp** mean absolute error: BS is not slightly miscalibrated, it has **the wrong shape**.

4.3 What the empirical curve is, and how it was built

The construction, in four steps

1. **Keep the theory that survives.** Diffusion says information about the terminal outcome scales with **σ√τ** — a 0.05% move with 30 seconds left is far more decisive than the same move with 5 minutes left. That part of Black–Scholes is *not* in dispute and is retained verbatim.
2. **Replace only the part that is empirically wrong** — the mapping from distance-to-strike onto probability. Instead of deriving it from a Gaussian, it is read off **observed market prices** as a piecewise-linear table of breakpoints.
3. **Normalise time out of the problem.** The raw distance is scaled by `√(refSec/τ)` to a common 300-second reference before the table is consulted, so one table serves every τ.
4. **Clamp and mirror.** The result is mirrored about the strike for downside moves and clamped to [1%, 99%] so the quoter can never be asked to price a certainty.

```
scaledΔ% = (S − K)/K × 100 × √(300/τ)   →   piecewise-linear table  →  P(YES)
```

```
const EMPIRICAL_BREAKPOINTS = [   // [ |Δ%| from strike , P(YES) % ]
  [0.000, 50], [0.005, 50], [0.02, 55], [0.05, 65],
  [0.10, 80],  [0.15, 94],  [0.30, 99],   // 0.30 extrapolated — see caveat
];

export function empiricalProbYes(spot, strike, tauSec, refSec = 300) {
  const tau            = Math.max(tauSec, 5);
  const rawDeltaPct    = ((spot − strike) / strike) * 100;
  const timeScale      = Math.sqrt(refSec / tau);        // retained BS σ√τ structure
  const scaledDeltaPct = rawDeltaPct * timeScale;
  const prob           = piecewiseLerp(Math.abs(scaledDeltaPct), EMPIRICAL_BREAKPOINTS);
  const yes            = scaledDeltaPct >= 0 ? prob : 100 − prob;
  return Math.min(0.99, Math.max(0.01, yes / 100));
}
```

Worked example

Spot $64,880, strike $64,850, 60 seconds to expiry:

```
rawDelta   = (64880 − 64850)/64850 x 100        = +0.0463%
timeScale  = sqrt(300/60)                       = 2.236
scaledDelta= 0.0463 x 2.236                     = +0.1035%
table      = lerp between (0.10, 80) and (0.15, 94)  = 80.98%
P(YES)                                          = 81.0%
```

The same $30 move with **300** seconds left scales to only 0.0463%, giving **P(YES) ≈ 64%**. Identical distance, very different conviction — which is precisely the behaviour BS's single σ could not produce across the whole curve.

Provenance — stated precisely

This is a calibration, not a first-principles derivation, and its lineage matters.

- The **breakpoints** were reverse-engineered from **live-observed Polymarket 5-minute BTC trading, via a third party's public write-up** — not from Polymarket's own published spec, and not from our own capture. The source file labels them "*a starting calibration, not verified ground truth.*"
- They were then **cross-checked against an independent second source** — live Kalshi order-book readings captured via headless browser automation — which is where the **35.9pp → 5.0pp** comparison comes from.
- This environment **cannot reach polymarket.com** (DNS blocked for the domain and all subdomains, confirmed via both curl and WebFetch), so the original source could not be re-verified directly from here.
- The final breakpoint (**0.30% → 99%**) is **extrapolated, not observed**. No source data exists past 0.15%. It is flagged as an assumption in the code and should be treated as one.

Anyone extending this curve to a new tenor or asset must re-observe real data — particularly beyond 0.15% — rather than trusting the extrapolated tail. Re-validation across distinct market regimes (trending vs chopping) remains the single most valuable outstanding piece of work on the pricing side.

Calibration against outcomes — the test that had never been run

Everything above validates the curve's **shape** against observed market *prices*. That is not the same as asking whether its probabilities are **right** — i.e. when it says 30%, does the event happen 30% of the time? That test was run on 31 July 2026 for the first time: **44,940 samples** of real BTC 1-second data resampled into **300-second windows** (BitBull's actual market horizon), sampled at τ = 270/240/180/120/60/30/15s.

Predicted band	n	Predicted	Actual	Error
0.00 – 0.05	5,805	2.0%	1.6%	–0.3pp
0.05 – 0.10	1,591	6.4%	8.4%	+2.1pp
0.10 – 0.25	3,011	18.2%	15.4%	–2.8pp
0.25 – 0.45	8,046	36.3%	26.9%	**–9.4pp**
0.45 – 0.55	9,746	50.0%	48.6%	–1.4pp
0.55 – 0.75	6,535	63.6%	69.9%	**+6.3pp**
0.75 – 0.90	2,549	82.0%	83.6%	+1.6pp
0.90 – 0.95	1,528	93.6%	92.6%	–1.0pp
0.95 – 1.01	6,129	98.1%	98.6%	+0.6pp
All extremes, folded	**15,053**	**97.1%**	**97.2%**	**+0.0pp**

At the extremes the curve is essentially perfect — +0.0pp over 15,053 samples. Any logic keyed off extreme fair value (the risk cap of §7.1, the directional gate of §7.2) is therefore acting on a trustworthy input. This was verified *before* narrowing the directional lockout, precisely because that change depends on it.

A new, unresolved finding: the mid-range is materially miscalibrated. The curve says 36% where reality is **26.9%**, and 64% where reality is **69.9%** — errors of –9.4pp and +6.3pp on large samples. The direction is consistent: **reality is more decisive than the curve predicts** in the middle of the distribution.

That is the same *under-reactivity* Black-Scholes was replaced for (§4.2), merely smaller, and it lives between the 0.02% and 0.10% breakpoints — interior points that were interpolated from few observations. Re-fitting those two breakpoints against outcome frequency is a concrete, cheap improvement and is now the highest-value outstanding item on the pricing side. Note this does **not** invalidate the 35.9pp → 5.0pp result, which measured error against market *prices*; it is a different and complementary test that had simply never been run.

Why it is better — and the honest limits

Dimension	Black–Scholes	Empirical curve
Accuracy vs market	35.9pp MAE	**5.0pp MAE — ~7× better**
Shape	One σ cannot be flat ATM *and* steep in the tails	Flat then steep by construction — matches observed trading
Time behaviour	σ√τ scaling (correct)	σ√τ scaling **retained unchanged**
Tail behaviour	Principled everywhere	**Extrapolated past 0.15%** — the weak point
Greeks	Closed-form δ and Γ	**None.** Piecewise-linear ⇒ Γ = 0 almost everywhere, undefined at knots
Provenance	Peer-reviewed, universally understood	Third-party observation + one independent cross-check

> **The system uses both — deliberately.** The empirical curve sets **fair value**, because it is 7× more accurate at the thing that decides whether a quote is exploitable. But it has **no usable derivatives**, so every risk quantity — the digital delta that sizes the perp hedge (§8.2), and the gamma that drives pin-risk widening (§6.2) — is still computed from **Black–Scholes**. BS was not deleted; it was **demoted from pricing to risk**, which is the job its smoothness actually suits it for.

4.4 An attempted fix that was abandoned — and what it uncovered

The mid-range miscalibration above looked like a three-number edit: measure the true outcome frequency at each breakpoint and write those values in. Over 128,400 samples that suggested 0.02% → 63.4 (from 55) and 0.05% → 73.0 (from 65). **It was not shipped**, because a check partway through showed the fix would have been actively harmful.

The red flag

The near-ATM bucket measured **54.3%**, when at essentially zero distance from the strike it must be ~50%. Directional drift was the obvious suspect and was ruled out: over the sample, P(a 300s window closes up) was **48.4%** — slightly *down*, not up.

The actual cause: √τ does not collapse the curve

The entire single-table design rests on one assumption — that after scaling distance by √τ, the probability is **independent of time to expiry**. Tested directly, it is not:

|scaled Δ|	τ 240–300s	τ 120–240s	τ 45–120s	τ < 45s
~0.02	61.4%	64.2%	64.3%	72.7%
~0.05	68.2%	73.8%	76.7%	81.2%
~0.10	**71.7%**	81.3%	85.6%	**89.7%**
~0.15	83.8%	88.7%	92.7%	96.3%

Each row should be flat. At ~0.10 the spread is **18 percentage points**. Since the same scaled distance means different things at different τ, **no single table can be correct across τ** — and fitting one to pooled data would have encoded a τ-average as though it were τ-independent.

> **This is the Black-Scholes failure, one level up.** BS could not fit the curve with a single σ (§4.2). Our replacement cannot fit it with a single *table*. The same structural error reappeared in the fix for it.

Fitting the exponent instead — and why that is not shipped either

Generalising the time term to `(300/τ)^α` and choosing α to minimise τ-dependence:

α	Mean τ-spread (train)	Mean τ-spread (holdout)
0.50 (current, theoretical)	13.70pp	17.54pp
0.70	8.00pp	14.41pp
0.88 (best on train)	**4.27pp**	11.96pp
1.10 (best on holdout)	—	**~11.9pp**

> **Robust:** α = 0.50 is definitively too low — it is the *worst* value tested on both samples, and the direction of the finding is unambiguous.
> **Not robust:** the optimum is unstable between samples (0.88 vs 1.10), and holdout τ-spread stays high (~12pp) at every α — that hour contained a strong directional episode and does not collapse well under any exponent.
>
> **Nothing was changed in production.** Two samples of a few hours each is not sufficient evidence to alter the core time-scaling of live pricing. Recorded as a documented finding rather than a shipped change.

What it actually needs: multi-day 1-second data spanning calm, trending and choppy regimes → fit α on that → *then* re-fit the breakpoint table conditional on the corrected α. If one exponent still fails to collapse the curve, the honest answer is a two-dimensional surface in (distance, τ) rather than a single table.

4.5 A second-order bug caught before shipping

> The application server computed its **own separate** Black–Scholes value for realized-spread and adverse-selection accounting. Switching only the **quoting** side to the empirical curve while leaving the **accounting** side on Black–Scholes would have made the two disagree **by construction** — manufacturing a fake **~30 percentage point "edge"** in the P&L dashboard that would have looked like trading skill but was pure model mismatch.
>
> Caught before shipping; both curves are switched together behind a single toggle. **This is the failure mode worth internalising:** when a model is used in two places, changing one place fabricates performance.

5 · Market Making — LMSR, Stoikov, and the Break-Even Identity

5.1 Research simulator

Before touching anything real, a prediction-market simulator was built in TypeScript/React: three AMM engines (LMSR, CPMM, LS-LMSR), an Avellaneda–Stoikov quoting overlay, rolling markets across 5/10/30-minute tenors with a real order book, and a **~95-agent behavioural trader population** — Poisson arrivals, heavy-tailed order sizes, and distinct archetypes (favorite-longshot-biased, momentum, contrarian, informed). Agents hold finite wallets that act as a reward function: they go broke and drop out, winners size up. Fully seeded and reproducible.

5.2 The P&L identity

The central analytical result — the market maker's profit decomposes exactly as:

```
net = spread capture − (LMSR subsidy + adverse selection) + hedge P&L − fees − funding
```

> **The inventory loss is structural, not a tuning problem.** Every market that resolves decisively costs the house approximately **b·ln2**, where b is the LMSR liquidity parameter. This is the subsidy paid for inventory-based price discovery. **It does not shrink with more volume**, because noise trading is symmetric and produces roughly zero net price displacement.

Consequences that follow directly:

- **Spread capture is the only revenue.** Break-even requires the vig to out-earn b·ln2 per market *plus* adverse selection.
- The break-even half-spread is roughly **scale-invariant in b** (volume scales with b too), so the real levers are **spread width** and the **ratio of noise flow to informed flow** — not liquidity depth.
- At GameBull's production b = 500, the subsidy is **b·ln2 ≈ $346 per market**, requiring roughly **17,000 contracts at a 2¢ vig** just to break even.

5.3 Empirical confirmation

The original default configuration **lost ~$17k over 8,000 ticks**: spread capture (~$6k) covered only about **25%** of the subsidy (~$24k). Tightening the Stoikov risk-aversion parameter k from 60 → 12 (i.e. wider quotes) moved the book to break-even in 95% of 5-minute windows.

6 · The Gamma Wall — The Core Quantitative Finding

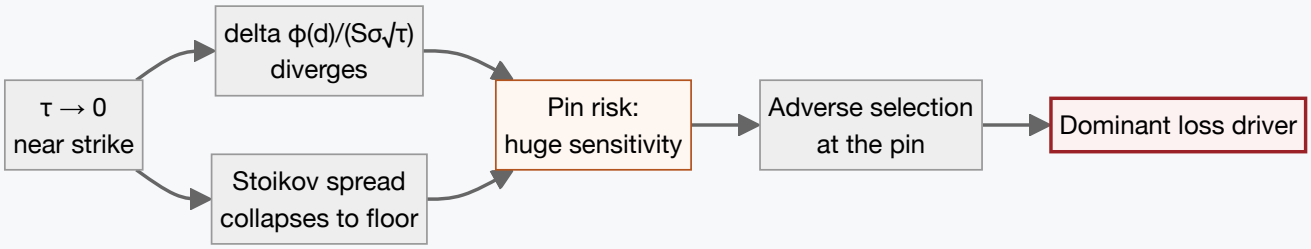
6.1 The mathematics

A 5-minute binary is **structurally short gamma**. Recall:

```
∂p/∂S = φ(d) / (S·σ√τ)   →   ∞ as τ → 0 near the strike
```

Three facts compound into the single worst regime in the product:

1. **Delta diverges** as τ → 0 at the strike — the position's sensitivity explodes precisely when there is no time left to react.
2. **The payoff is a step function** — no linear instrument can hedge a discontinuity.
3. **The Stoikov spread naturally collapses** to its floor as τ → 0, because inventory risk over a vanishing horizon looks small — **exactly when adverse selection is worst**. The model actively quotes tightest at the most dangerous moment.



6.2 The fix — in quoting and risk, not hedging

Because no linear hedge can address a discontinuity (§8.1), the problem was solved on the **quoting** side instead:

(a) Pin-risk spread widening

```
// drivers/lib/quoting.mjs
const gammaWiden = qp.gammaWiden ?? 0.03;
// p(1−p) peaks ATM (max outcome uncertainty); /sqrt(tauHat) surges into expiry
const pinRisk = gammaWiden * pFair * (1 − pFair) / Math.sqrt(tauHat);
halfSpread = base + pinRisk + invWidenTerm + flowWidenTerm;
```

The term **p(1−p)/√τ** is deliberately shaped to mirror where gamma actually lives: `p(1−p)` peaks at-the-money where outcome uncertainty is greatest, and `1/√τ` surges into expiry. It widens quotes exactly where the risk is, and stays out of the way elsewhere.

(b) Final-30-second reduce-only lockout

Inside the last 30 seconds the maker refuses **inventory-increasing** trades. It will let a user close, not open, at the wall.

6.3 Measured effect (A/B, staged)

Configuration	Mean / window	Profitable windows	Worst window	Std dev
Baseline (no gamma handling)	$50.7	78%	–$74	84
+ gamma wall fix alone	**$71.5**	**95%**	–$56	**54** (–36%)
+ inventory & risk-tier layers	$77.9	97%	–$43	49 (–42%)

Stated deliberately as a staged result: the p(1–p)/√τ term **alone** accounts for 78%→95% and –36% volatility. The 97% / –42% figures require three further levers (inventory-proportional widening, a risk-tiered hedge dial, and a longer per-tenor lockout). Attributing the full number to the single formula would overstate it.

7 · Risk Management

7.1 Quote caps from an explicit risk:reward ratio

Quoting a favourite at 99¢ means risking 99¢ to win 1¢ — a 99:1 ratio. Rather than pick a cap by feel, it is derived:

$$cap = R \cdot 100 / (R + 1)$$

```
const MAX_RISK_REWARD_RATIO = 19;                    // → 95¢ general cap
const AGGRESSIVE_CAP_CENTS  = Math.round((R*100)/(R+1));
const NEAR_EXPIRY_MAX_RISK_REWARD_RATIO = 49;        // → 98¢, final 10s only
```

Above the cap the maker **stops adding to the favoured side** and instead bids the underdog directly at its own fair price (≤5¢ by construction) — a materially better risk:reward trade at that point.

> **Why not 99¢ like the real platforms?** A deliberate, defensible refusal. Kalshi and Polymarket support 1¢ quoting because they have **many competing market makers, diversified across many concurrent markets**. BitBull is a **single house, on one market at a time, with limited model validation**. A 99:1 risk:reward ratio is not justified confidence at this stage. The narrow R=49 exception applies only inside the final 10 seconds — no time for a reversal, and the only regime where the empirical curve has real validation data.

7.2 Directional lockout — and why it was narrowed

When fair value becomes extreme, the maker bids the **favoured** side aggressively and posts **no bid at all** on the underdog. Because the matcher pairs a YES bid against a NO bid at complementary prices, the favoured-side bid is itself what makes the underdog cheap to buy — a separate underdog bid is not needed for that. Observed live:

```
[mmp] ⏸ directional (τ=37s, fairYes=5.1%, spot $64840 vs K $64880) — NO favored
[mmp] favored NO capped at 95¢ (fair implied 95.1¢, R=19:1 risk:reward limit)
[mmp] YES side skipped (directional lockout — underdog, no direct bid needed)
```

The consequence, stated plainly: while this is active users can buy *only* the cheap underdog. They cannot take the favourite at all.

How broadly it was firing

Measured over 1,062 live samples, directional mode was active far from expiry:

τ while lockout active	Share of samples
> 60s	**47%**
> 120s	18%
Earliest observed	**τ = 213s** of a 300s market

Three reasons that was too broad

1. **Single-maker venues are different.** Market makers on Kalshi and Polymarket withdraw quotes when risk:reward turns bad — that is ordinary, and no maker is obliged to quote. But those venues have **many** makers, so one stepping back is competition. Here the house is the **only** maker, so stepping back removes that side of the market outright. Identical behaviour, materially different consequence.
2. **The risk arithmetic ran opposite to the stated rationale.** The design kept the favoured side on the reasoning that it is "statistically likely to pay out." True — but that is a statement about *frequency*, not risk. On a measured episode (fairYes 98%, 3,448 shares at 98¢):

	Permitted then	Blocked then
House ends up	long the **favourite** @ 98¢	long the **underdog** @ 2¢
Max loss	**$3,379** (2% chance)	**$69** (98% chance)
Max gain	$69	$3,379
Risk : reward	**49 : 1 against the house**	0.02 : 1
EV at fair value	~0	~0

   Both are EV-neutral at fair value — the vig is what makes either profitable. The difference is **skew**, and the side being retained was the **negatively skewed** one: small frequent gains against a rare catastrophic loss. That is precisely the tail the gamma widening (§6), expiry lockout, and perp hedging (§8) all exist to contain. Worse, holding *both* sides is a complete set worth a guaranteed 100¢ — **two-sided quoting is self-hedging; one-sided quoting concentrates risk.**
3. **Redundant mid-market.** The R=19 → 95¢ cap (§7.1) already prevents the house paying silly prices for a favourite, without removing user access.

> **Is this manipulation? No.** Manipulation means creating artificial prices — spoofing, wash trading, marking the close. Declining to quote is **withdrawal of liquidity**, which every maker is entitled to do; even designated market makers carry bounded obligations, not unlimited ones. The interface also greys the side out and states "No liquidity" rather than displaying a phantom price, so nothing is concealed.
>
> **But "not manipulation" is not the same as good practice.** On a single-maker venue, and combined with the favourite-longshot bias (Kalshi buyers of ≤10¢ contracts lose >60% on average), offering users *only* the historically-losing side for minutes at a time is difficult to defend — to a user or on risk grounds.

The change

Directional mode is now **gated on time as well as fair value**: it fires only inside the final **30 seconds** (`MMP_DIRECTIONAL_MAX_TAU`, rollback by setting it to 99999 — no code edit). Near expiry the justification is genuinely strong: no time for a reversal, and it is exactly the pin-risk regime. Outside that window both sides quote regardless of how extreme fair value is.

Precondition, checked first: this change trusts fair value to decide when a market is extreme, so the curve's calibration at the extremes was verified before shipping it — **+0.0pp error over 15,053 samples** (§4). **Verified live:** at τ=274s with fairYes=42% the book now shows YES 12,500 shares and NO 11,282 shares, bid 40.1 / ask 43.4 — both sides tradeable, where previously one side could go dark for up to 213 seconds.

7.3 Tick-size upgrade — a real defect in the matching engine

The request was to move from a 1¢ tick to 0.01¢. Before implementing, the production matching engine was read, revealing a genuine floating-point defect: it computes `parseInt(bidAmount * 100)` — float multiplication followed by **truncation rather than rounding**.

Quantified empirically across all 9,999 two-decimal cent values:

Tick increment	Corrupting values	Verdict
0.01¢ (as requested)	**573 / 9,999 (5.7%)** — e.g. 99.99 → 99.98	Rejected
0.1¢ / 0.25¢ / 0.5¢	**0**	**0.1¢ shipped**

The investigation also surfaced float display noise elsewhere (values such as `52.900000000000006`), fixed at source in the tick-snapping function.

7.4 How much do user trades move the price?

Inventory reaches the quote through two channels — the reservation price shifts, and the spread widens:

```
shift = (netSkew / b) × γ × σ² × (τ/300)   ·   widening = invWiden × |netSkew / b|
```

With live parameters (γ=1.0, **σ=0.10**, invWiden=0.02, b≈18,005):

User buys	Mid shift (τ=150s)	Spread widening	Total
100	0.003¢	0.011¢	0.014¢
1,000	0.028¢	0.111¢	0.139¢
5,000	0.139¢	0.555¢	0.694¢
12,500 (the inventory cap)	**0.347¢**	1.389¢	1.736¢

Price impact is effectively nil at realistic sizes. Even a *maxed-out* position moves the mid **0.35¢ — about 3½ ticks** against a ~1.5¢ half-spread. A 500-contract trade moves it 0.014¢, a seventh of one tick, so it does not change the displayed price at all. The dominant cause is σ = 0.10, which makes the σ² term **0.01** and suppresses the whole effect by two orders of magnitude. Three further observations:

- **The spread widens ~4× more than the mid moves.** The inventory response is mostly "quote wider," not "quote skewed" — which protects the house but does nothing to *attract* the offsetting flow that would flatten the book.
- **It decays into expiry.** The τ/300 term means impact at τ=30s is a fifth of its value at market open — weakest exactly when inventory is most dangerous.
- The LMSR/Stoikov inventory-skew machinery is therefore doing almost no visible work at current sizes.

What the right answer probably is

A crypto binary is structurally different from a political market. On an election market, order flow **is** the information — there is no external oracle, so a buyer genuinely moves the fair estimate. On a BTC binary the truth is set on Binance and Coinbase with billions in volume; a few dozen users here carry **essentially no information** about where BTC goes.

So price impact should not exist for *price discovery*. It exists for one job: **inventory management** — making it progressively more attractive for the next user to take the other side. Judged against that purpose the current setting is too weak to function. The lever is σ (`MMP_SIGMA_P`): raising it 0.10 → 0.30 multiplies the mid shift 9× without touching the spread logic. That should be validated in the simulator before going live — it has not been.

8 · Hedging — And Where the Mathematics Refuses to Cooperate

8.1 The fundamental result NEGATIVE

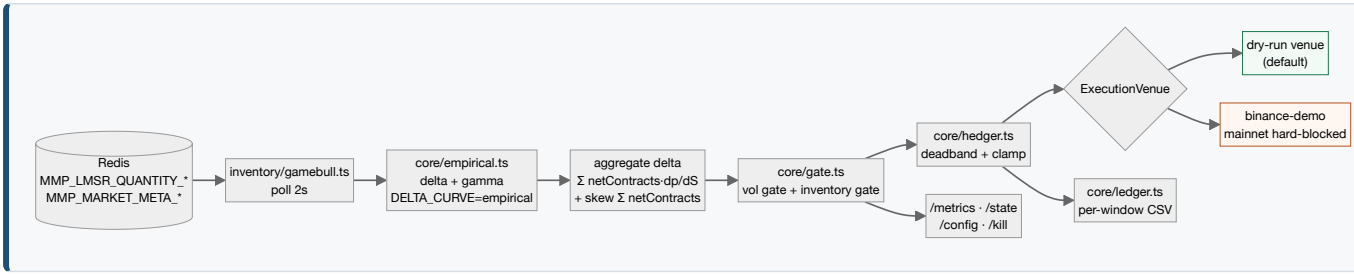
> **A perpetual future provably cannot hedge a binary's terminal gamma.**
>
> A perp is **linear**: Γ ≡ 0 by definition. A digital's value near expiry is a **step**, with delta diverging as φ(d)/(Sσ√τ). No linear instrument can replicate a discontinuity — this is a mathematical impossibility, not an engineering shortfall.
>
> **Confirmed empirically in an A/B test:** worst-case window went from **–$94.5** → **–$94.5** — a delta of **exactly 0.0**, with hedging cost ≈ benefit. Risk-removed-per-dollar-spent ≈ 0.

This is the single most important negative result in the project, and it is **load-bearing**: it is precisely why §6's fix had to live in quoting rather than hedging, and why §9.2 exists at all.

8.2 What perps *can* do — and the sidecar that does it

Perps hedge **delta** — directional exposure — very effectively. That is worth capturing:

Metric (5 windows × 5 min, real Binance paths, $10k budget)	Result
P&L dispersion removed (delta / combined hedge)	**33–37%**
Worst-case window	**–$115 → +$85 / +$109**
Best-performing mode	Combined (delta + sentiment tilt)
Irreducible residual	Adverse selection — mathematically unhedgeable; only the vig pays for it



Sign mapping — the silent-failure detail

> The hedge direction maps as **qYes = houseNo, qNo = houseYes**, so that `(qYes − qNo)·dp/dS` **offsets** house exposure rather than amplifying it.
>
> Getting this backwards **would not fail loudly** — it would silently **double** the risk while appearing to hedge. It was therefore verified end-to-end against live order flow rather than reasoned about on paper. Live-verified again during this document's preparation: *user buys NO → house long YES → delta –0.0889 → SHORT hedge –0.0463 BTC.* Correct.

Safety architecture

- **Read-only.** Reads Redis inventory keys; never writes to any GameBull data store.
- **Mainnet hard-blocked** at construction — the service refuses to start against a mainnet Binance host.
- **DRY_RUN by default**, with an explicit enable gate and a max-position cap.
- **Control plane:** Prometheus `/metrics`, `POST /config` for runtime reconfiguration without redeploy, idempotent `POST /kill` (flatten and disable).
- Secrets only in a gitignored `.env`; containerised, non-root, health-checked.

8.3 The $16M problem UNRESOLVED

> **A structural risk mismatch, surfaced rather than solved.**
>
> Inventory limit: **12,500 contracts**. Hedging budget: **$10,000**.
> At-the-money near expiry, the worst-case implied notional required to hedge is approximately **$16 MILLION** — because the digital's delta explodes as $\tau \to 0$.
>
> A delta-aware dynamic cap was implemented and live-tested. It **collapsed the quotable size to 13–180 contracts** under entirely ordinary conditions, and was rolled back to default-off.
>
> **This was reported as a finding, not a bug.** The formula was correct; it revealed that the hedge budget is drastically undersized relative to real gamma. Resolving it is a **capital allocation decision**, not an engineering one — and it remains open.

8.4 Fee revenue is structurally zero OPEN

In market-only mode **every fill is house-versus-user**, so there is no user-to-user trade on which to charge a fee. The Hedge Desk correctly displays **Fee revenue $0.00**. Real fee revenue requires enabling limit orders so users can trade with each other — the order-entry path for this already exists.

8.5 The hedge was sized off the wrong curve DEFECT FIXED

Found when a reviewer observed that quoted odds plainly did not move as much per dollar of BTC as the reported delta implied. They were right, and the cause was structural.

> **The exchange quotes on the empirical curve (§4.3). The hedging service was sizing its hedge from the Black–Scholes derivative.**
>
> Those are different curves, so the hedge was sized against a sensitivity the book **does not have**.

Measured against 78 live spot moves ≥ $2 recorded in `data/driver.log`:

τ	Observed d(fairYes)/d(spot)	Black–Scholes claimed	Over-hedge
200–300 s	0.55 ¢ / $	0.90 ¢ / $	**1.6×**
100–200 s	0.68 ¢ / $	1.27 ¢ / $	**1.9×**
< 100 s	0.93 ¢ / $	2.02 ¢ / $	**2.2×**

At the money the disagreement reaches **3.15×**. Critically, the correction is **not** a uniform "hedge less": for an out-of-the-money position near expiry it reverses — at $40 OTM with 60 s left, Black–Scholes sized $297k where the empirical curve requires $765k, i.e. it was **under**-hedging by 2.6× exactly where pin risk concentrates. This is a correctness fix, not a cost saving.

> **This is the same defect class as §4.5** — a number derived from one price curve while the system quotes on another. That instance was caught before shipping; this one was not, and survived in the hedging path. The generalised lesson has been adopted as a rule: **any figure derived from a price curve must record which curve produced it.** `/state` and the Hedge Desk now both report `deltaCurve`.

Why the derivative is numerical, not analytic

The empirical curve is piecewise-linear, so its exact derivative is a step function — discontinuous at every breakpoint, and **exactly zero** inside the flat [0.000%, 0.005%] segment straddling the strike. Used directly it would churn the hedge on fitting artifacts and drop the hedge to zero at the point of maximum risk. `src/core/empirical.ts` therefore takes a central difference over a finite bandwidth ($0.5 \cdot \sigma\sqrt{\tau}$, floored at $5), which averages the slope across the region spot will plausibly visit — also the variance-minimising hedge ratio for a kinked curve. Rollback is `DELTA_CURVE=bs`.

> **Maintenance hazard, recorded deliberately.** `EMPIRICAL_BREAKPOINTS` now exists in *two* repositories — `drivers/lib/pricing.mjs` and the hedging service's `core/empirical.ts`. If they drift apart, the hedge silently reverts to being sized off a curve the exchange no longer quotes, and **nothing fails loudly**. `test/empirical.test.ts` pins the values as a tripwire.

Skew is a contract count, not a dollar figure

The Hedge Desk displayed **$66.8M of "skew" against roughly $1,000 of user flow**. Both numbers were real; the label was wrong. Two distinct quantities had been collapsed under one word:

Quantity	Definition	Units
Inventory skew	Σ(qYes − qNo) — which side the book leans, and what the hedge exists to flatten	contracts
Delta exposure	Σ(qYes − qNo)·dp/dS — what that lean is *worth* per $1 of BTC	BTC-equivalent

The panel now reports net contracts (with gross alongside, so an offsetting book is visible as offsetting rather than accidentally flat) and the δ separately, each labelled with its curve.

8.6 The pin case where both legs lose STRUCTURAL

A scenario raised during review: users buy YES heavily while spot sits above the strike; the house takes the other side and hedges; settlement lands still above the strike but lower than entry — and **both** the inventory and the hedge lose. Reproduced exactly (K = 63,800, spot K+10, τ = 300 s, 1,786 contracts, hedge long 8.40 BTC):

Settlement	Outcome	Inventory P&L	Hedge P&L	Total
K − 11	NO wins	+$957	−$88	**+$869**
K + 4	YES wins	−$829	−$48	**−$877** ← both legs lose
K + 30	YES wins	−$829	+$239	−$590

It occurs in a specific band — spot moves *toward* the strike without *crossing* it, so the hedge loses on direction while the binary still resolves against the house. Frequency in that configuration: **4.1%** of outcomes.

> This is **not** a hedging defect. It is §8.1's terminal gamma stated in P&L terms: the payoff jumps $1 at the strike, the perp is linear, and no linear instrument spans a step. Hedging remains net beneficial here — it removes **38%** of P&L dispersion ($890 → $551 SD). The double-loss band is the premium paid for the 96% of paths where the hedge works, and it cannot be removed by sizing, timing, or holding.

A rejected proposal, and why

It was proposed that losing perp legs simply be held rather than closed — "mean reversion", so the hedge never realises a loss. Tested on 27 days of real BTC 1-minute data, 20,000 simulated legs:

Policy	Win rate	Median	Mean	Worst leg	SD
Close at expiry (current)	50.9%	+$1.01	**+$0.03**	−$697	$69
Hold until green (1 h cap)	87.9%	+$16.78	+$0.71	−$1,419	$124
Hold until green (24 h cap)	**97.3%**	+$19.55	**−$3.23**	**−$3,297**	$263

The rule delivers a 97% win rate and *loses money*: mean P&L turns negative while the worst leg degrades **4.7×** and volatility nearly **4×**. 2.7% of legs never turn green and pay for all the wins. Two further objections are specific to this system: a red perp leg is, by construction, the event in which the *inventory* gained — the two are anticorrelated by design — and the binary settles in five minutes while the perp does not, so a leg held past settlement is no longer a hedge but a naked directional position sized off an exposure that no longer exists.

8.7 Hedge economics — the finding that constrains everything above CRITICAL

> **At full delta size, the perp hedge costs more in exchange fees than the spread earns.**
>
> Neutralising 1,786 contracts requires **$536,000** of perp notional — **300× the $1,786 maximum possible loss** on the position. Fees scale with the *perp* notional; revenue scales with the *binary* notional.

Quantity	Value
Full hedge notional (1,786 contracts, ATM, τ=300 s)	$536,004
Taker fee, enter + exit, per 5-minute window	**$429**
Gross edge at a 3¢ spread	$54
Break-even spread required	**24¢ per contract** (on a ~54¢ contract)
Fees at full hedge, 288 windows/day	$123,495 / day

Observed spreads are 3–6¢, against a 24¢ break-even. **Full delta hedging is therefore uneconomic at taker fees** — independent of direction, vol, or curve.

> The live ledger shows only $25.11 of hedge fees across 465 windows because `armed_frac` is **1.8%** and the $10,000 notional cap holds the position at roughly 1% of full size. **The cap of §8.3 has been inadvertently shielding the system from this cost.** Raising it to "fix" the $16M gap — the obvious reading of §8.3 — would walk directly into the fee wall. The capital question and the fee question must be answered together.

The hedge-fraction frontier

Risk reduction is strongly **concave** in hedge size, which is what makes a partial hedge the rational choice rather than a compromise:

Hedge fraction	Risk removed	Taker fees / window	Maker fees / window
0.10	7%	$43	$21
0.30	**20%**	**$129**	**$64**
0.50	31%	$214	$107
1.00	38%	$429	$214

The first ~30% of the hedge buys two-thirds of the total available benefit at under a third of the cost. Above 0.75 it is actively counterproductive — dispersion rises from $546 to $551, the over-hedge predicted by payoff-matched sizing.

Consequent direction: hedge a fraction rather than full delta; move from taker to maker execution (an immediate halving); and source profit from the spread rather than the hedge — the hedge never adds expectancy, it only removes variance. Terminal gamma at the pin is priced into spread, not hedged.

9 · Work In Progress

9.1 Kronos — can a learned model beat the empirical curve? IN PROGRESS

Kronos is an open-source foundation model for financial candlestick sequences. The question: can a fine-tuned Kronos beat the hand-built empirical curve as BitBull's pricing source?

Method

Monte Carlo. Take the last 512 candles as context, set an at-the-money strike (mirroring market open), draw N independent samples of the future close, and take **P(YES) = fraction of samples ≥ strike**. Compared against the real outcome and against `empiricalProbYes` **ported verbatim** from production — the literal live breakpoints, not a re-derivation. Inference latency is recorded as a first-class metric, since it is the true gate on ever using this inline with the ~2s quote loop.

Model	Params	Training	Best val loss	Status
Kronos-mini @ 5m	4.1M	866 min	2.7551	Done
Kronos-mini @ 1m	4.1M	109 min	2.5153	Done
Kronos-mini @ 1s	4.1M	515 min	1.3444	Done
Kronos-small @ 1s	**24.7M**	Kaggle T4	—	**Running**

An evaluation bug found while scoring Kronos-small — and what it invalidated

> **Every Kronos number produced before 31 July 2026 was measured with dropout active at inference.**
>
> Scoring Kronos-small immediately crashed on Apple MPS with `scaled_dot_product_attention for MPS does not support dropout`. Tracing it revealed that `KronosPredictor` wraps generation in `torch.no_grad()` — which disables **gradients** but **not dropout** — and **never calls** `.eval()`.
>
> Kronos-mini did not crash because its `attn_dropout_p` is 0.0. But its `ffn_dropout_p` and `resid_dropout_p` are both 0.2, and those **were live on every forward pass**. Each Monte-Carlo draw therefore carried random dropout noise on top of genuine sampling entropy — accidental **MC-Dropout**. Kronos-small has `attn_dropout_p = 0.1`, which additionally hits the MPS restriction; that crash is the only reason the bug surfaced at all.
>
> Fixed with explicit `model.eval()` / `tokenizer.eval()` plus a mode assertion, and **both models re-scored** on the corrected harness.

Result at 1 second — corrected, and it reverses a prior conclusion

The empirical curve's numbers are **bit-identical** across the two runs (Brier 0.1618, MAE 36.4pp), which confirms the harness is otherwise deterministic and only the model path changed — a useful built-in control.

Model	Brier ↓	MAE ↓	Var(error)	Misses >80pp
Kronos-mini (dropout active — **invalid**)	0.2050	27.2 pp	0.1309	24 / 150
Kronos-mini (corrected)	**0.2361**	**29.7 pp**	0.1481	**31 / 150**
Empirical curve (unchanged, control)	**0.1618**	36.4 pp	**0.0295**	**0 / 150**
50/50 blend (corrected)	0.1766	33.0 pp	0.0676	4 / 150
Baseline — always 0.5	0.2500	—	—	0 / 150

Two conclusions I previously stated are now withdrawn.

- **"The blend beats both models." False under correct inference.** The blend scores 0.1766 against the empirical curve's 0.1618 — the **empirical curve alone is now the best model on Brier**. The blend's apparent advantage was an artefact of dropout noise.
- **"Dropout noise was inflating the confident-wrong tail." Wrong in direction.** Removing dropout made the tail *worse* — catastrophic misses rose from 24 to **31 of 150**. The noise had been pulling predictions toward 0.5, acting as a crude uncertainty regulariser that *flattered* calibration. A sharper model is a more confidently wrong one.

What survives correction:

- **Kronos still wins MAE** (29.7pp vs 36.4pp) — it lands closer on average.
- **Kronos still loses Brier decisively** (0.2361 vs 0.1618), and its error variance is **5.0× higher** (0.1481 vs 0.0295). The `Brier = MAE² + Var(error)` decomposition still explains the split exactly: Kronos's MAE² is lower, but its variance is not.
- **Skill remains concentrated**, and sharpened: in the 0.02–0.05% moneyness bucket Kronos now scores Brier **0.0251** vs empirical 0.0463 (MAE 2.8pp vs 17.7pp). Near-ATM it is far worse (0.4056 vs 0.2472).
- **Raw forecasting skill is still absent.** Kronos beats a naive no-change forecast on only **38.0%** of points, with a **41.3%** directional hit rate — both worse than a coin flip.

The honest read. Kronos-mini is *usually closer* but *occasionally catastrophically wrong*; the hand-built empirical curve is *consistently mediocre* and **never wrong by more than 71pp**. For **quoting**, MAE is the relevant loss and Kronos looks attractive. For **risk sizing**, Brier is the relevant loss and the empirical curve wins outright. Given that this platform's dominant failure mode is confident mispricing at the pin (§6), **the empirical curve remains the correct production choice** on this evidence.

A hypothesis tested and rejected — with a caveat of its own

Before the eval-mode bug was found, the confident-wrong tail was tested for a simpler explanation: with only 6 MC samples, P(YES) can take just 7 discrete values, so one unlucky draw could masquerade as overconfidence. This was probed directly — draw 40 samples once, then recompute P(YES) from the first k for k $\in$ {6, 12, 20, 40}. The count of catastrophic misses held at **exactly 6 at every sample size**, rejecting sampling resolution as the cause.

That test is now itself suspect. It ran on the **uncorrected** harness, so dropout noise was present throughout and the experiment could not have isolated it. The conclusion — that resolution is not the culprit — is probably still right, since the miss count was perfectly flat in k. But it tested the wrong variable and should be re-run on the corrected harness before being relied on.

Kronos-small: the capacity test, resolved

Trained on a Kaggle T4: **24,741,376 parameters** (exactly 6.0× mini), tokenizer val loss 0.0016, basemodel val loss **1.5819**, in **33.8 minutes** versus 8.6 hours for mini locally. Scored through the **identical** corrected harness — same 25 windows, same 6 offsets, same holdout — so the comparison is like-for-like.

Model	Brier ↓	MAE ↓	Var(error)	Misses >80pp
Kronos-mini (corrected)	0.2361	29.7 pp	0.1481	31 / 150
Kronos-small	**0.1900**	**23.3 pp**	0.1356	**25 / 150**
Empirical curve	0.1618	36.4 pp	**0.0295**	**0 / 150**
Blend — small + empirical	**0.1498 ← best**	29.9 pp	0.0607	4 / 150
Baseline — always 0.5	0.2500	—	—	0 / 150

Capacity helped, and the qualitative shift matters more than the numbers.

- Small beats mini on **every** measure — Brier, MAE, and catastrophic misses (31 → 25).
- **It has genuine forecasting skill, which mini did not.** Directional hit rate **57.3%** versus mini's 41.3%, and it beats a naive no-change forecast on **50.0%** of points versus mini's 38.0%. Mini was *worse* than a coin flip; small is meaningfully better than one. This is the result that justifies the experiment.
- The **blend is best overall** (0.1498 vs the empirical curve's 0.1618).

Three caveats, none of them small.

1. **Kronos-small alone still loses to the empirical curve on Brier** (0.1900 vs 0.1618) — its error variance remains 4.6× higher. Usually closer, occasionally badly wrong.
2. **Latency is probably disqualifying for inline use:** p50 **1,756ms** per single sample versus mini's 548ms. Six samples per quote is roughly 10 seconds against a 400ms quote loop. Any production use would have to be out-of-band — precomputed, or on faster hardware — not inside the quoting path. (A 388-second maximum in the latency log is an artefact of the machine sleeping mid-run, not a real measurement.)
3. **The "blend wins" conclusion has now flipped twice** — it held under the buggy harness, failed once corrected on mini, and holds again with small. n=150. It should be reproduced on a materially larger sample before anything is built on it.

A confound worth stating: Kronos-small's tokenizer caps context at 512, forcing a lookback of **192** against mini's **1024** — 6× the parameters but roughly one-fifth the history. Small won *despite* that handicap, which strengthens the capacity conclusion, but it means the two models were not given equal information.

Infrastructure note: local training contended for RAM with the live exchange (16GB machine; swap hit 97%, degrading quoting — see §10.2). Kaggle's P100 was tried first and **failed hard**: compute capability sm_60 is below the sm_70 minimum of Kaggle's current PyTorch build. A T4 (sm_75) worked.

9.2 Cross-market gamma hedging NO-GO (on current data)

Since perps cannot hedge terminal gamma (§8.1), the question becomes: can the exchange's **other concurrently-open markets** hedge each other? They are the only other instruments on the same underlying with digital-shaped gamma.

What was built

- Closed-form digital **gamma** added to the pricing core, unit-tested for the τ→0 blowup and the sign flip across the strike (the property that makes naive |Γ|-only hedge ratios unsafe).
- Per-market gamma computed live in the inventory poll.
- A CSV exposure recorder logging `(tick, market, τ, spot, qYes, qNo, delta, gamma)`, behind a `RECORD_EXPOSURE` flag, off by default.
- An offline analysis script with **pre-registered kill criteria** — thresholds fixed *before* seeing results.

Measurement	Result	Kill threshold	Verdict
Gamma concentration (gross/net)	p50 = 1.02	kill if p50 < 1.15	**FAIL**
Hedge availability — staggered same-tenor	24.3%	kill if < 30%	**FAIL**
Hedge availability — long-tenor ladder	30.0%	kill if < 30%	PASS (borderline)
Flow decorrelation (median |corr|)	0.023	kill if > 0.8	PASS

Verdict: NO-GO. Two of three measurements failed pre-registered kill criteria. Concurrent markets do not reliably have usable gamma available at the moments it is actually needed.

A bug caught by distrusting a clean result. The first run reported **0.0% hedge availability on both legs** — suspiciously tidy. Investigation found the usable-gamma band (`c·σ√τ` , a **fractional** log-moneyness width) was being compared directly against `|spot − strike|` , a **dollar** distance. A ~0.001 fractional band can never exceed a multi-dollar gap, so the comparison returned `false` unconditionally — *independent of whether real availability existed*. Fixed by converting units before comparing.

Read the NO-GO honestly. This run was a short, hand-seeded smoke test — **61 ticks, 70 high-gamma moments, 2 market pairs**. That is far too small to settle the research question; its real value was proving the **pipeline computes correctly** post-fix. Two specifics flagged rather than glossed:

- The long-tenor ladder passing is the **opposite** of what σ√τ theory predicts for a mature market — a long-dated strike should go stale as spot drifts over hours. The 1h market simply had not had wall-clock time to drift, so its band was artificially "fresh".
- Gamma concentration reading ~1.0 versus an earlier same-tenor run's 2.7–2.8 is an unexplained shift — plausibly small-sample noise at n=61, but not waved away.

Real next step: leave the 5m/15m/1h ladder running for hours/days under organic flow, then re-run the **identical, already-built** script. Cheap — it needs wall-clock time, not new code.

9.3 Options overlay — the remaining candidate PROPOSED

With cross-market hedging returning NO-GO, options are the only remaining instrument that can address terminal gamma. A digital paying $1 if S ≥ K is replicable as a tight bull call spread:

```
digital ≈ [ call(K−ε) − call(K+ε) ] / (2ε)
```

So a **short digital is hedged with a long call spread at the strike** — matching both the terminal payoff **and** the near-strike convexity, because the spread's delta self-adjusts the way the digital's does. This is the correct instrument *shape*, unlike a perp.

Obstacle	Detail
Expiry granularity	Shortest listed BTC options (Deribit 0DTE) are **daily**; nothing matches a 5-minute window.
Strike granularity	Deribit strikes are spaced ~$1,000 apart vs BitBull's ~$100 ATM strikes — often no listed strike near K±ε.
Theta	Short-dated premium likely exceeds the perp hedging cost (~$542 total in the A/B).

Realistic architecture — both, split by job: perps continue covering delta (cheap, linear, continuous); a **small option overlay at the busiest strikes** covers gamma/tail risk, accepting basis risk against the nearest listed expiry rather than neutralising each individual window.

> **The trade-off inverts versus perps:** worst-case and max-drawdown **improve**, while **mean P&L worsens** — you pay theta. Whether that is worth buying depends on tail fatness under **real** directional flow. Near-flat simulated flow makes the overlay look like pure cost; genuine one-sided informed flow is where it earns its premium back.

Next concrete step: model call-spread replication against the **already-recorded** per-window ledger (inventory, spot paths, realized vol) and present perp-only versus perp+overlay on worst-window / max-drawdown / mean-P&L. This is an analysis task on existing data — **no new infrastructure required**.

10 · Engineering Record

10.1 Production defects found by reading their code

All 54 microservice repositories were mapped into product lines. Findings relevant to this vertical:

- **LMSR parity proven.** Their `calcLMSRPrice.pYes` equals our `sigmoid((qYes−qNo)/b)` to **1e-16**, and their parameter named `volatility` **is** the liquidity parameter b (default 500). This mapping was the prerequisite for sizing any hedge against live inventory.
- **No hedging exists anywhere** in their codebase — the gap this work fills.
- **Market metadata lacks strike / expiry / underlying** (risk is managed only via max-loss caps), so perp hedging requires a small additive metadata publish. This is the single ask of the platform team.
- **Rounding bias:** price-interval flooring uses `Math.ceil`, rounding quotes **up** a tick.
- **Float truncation** in bid amounts (§7.3).

10.2 The liquidity-gap investigation — including a corrected diagnosis

Users intermittently saw "no liquidity". A dedicated **liquidity watcher** was built (400ms polling, structured CSV of every gap with duration and τ), and later surfaced as a live tab in the app.

Root causes found and fixed:

1. **Process cold-start.** Quoting spawned a **fresh Node process every cycle** (fork/exec + V8 init + new DB/Redis handshakes). Converted to a **persistent worker** with an internal loop and auto-restart.
2. **Self-introduced race.** The conversion left a market-creation seed call that spawned a *second* uncoordinated quoter, racing the persistent one on new markets — confirmed live via back-to-back cancels with no repost, producing a 15s empty book. Removed.
3. **Duplicate market generator** left running from an earlier rollback, doubling per-cycle workload. Killed.
4. **Unbounded network call.** `placeBid()` had **no timeout** — under contention it could stall the entire sequential quote loop for 40+ seconds. Fixed with `AbortSignal.timeout(5000)` plus per-post and per-market error isolation.
5. **Watcher false positives.** It flagged the intentional **directional lockout** (§7.2) as an outage. Taught to check the opposite side's best price before alerting.

> **A diagnosis that was wrong, and the correction.** The residual gaps were initially attributed to **CPU contention** (load average 7.7–14.3 on 8 cores). That was challenged — CPU was largely idle. Re-investigation via `vm_stat` found the true mechanism was **RAM exhaustion and swap thrashing**: sustained 260–525 MB/s swap-out, with both the ML training process (11GB) and the Docker VM (7.4GB) in uninterruptible disk-wait on a 16GB machine, swap 94–98% full.
>
> **Generalisable lesson:** *idle CPU with ongoing stalls is the signature of I/O wait, not scheduling starvation.* Mitigation: Docker's VM allocation was reduced 8GB → 5GB (a VM-level restart, deliberately **not** `docker-compose down`, which would have destroyed unvolumed data).

10.3 Quoting latency — the residual gap, and the real fix

After §10.2's fixes the "no liquidity" flashes persisted at ~15.9/min. The cause was isolated by correlation rather than assumption:

> **92.3% of gap events began within the cancel window, median offset +0.000s.** The gap *was* the quoter's own cancel. 197 of 203 events were both-sides-simultaneous.
>
> **Resource exhaustion was explicitly ruled out:** Docker sat at 51% of its 4.8GB allocation and pageouts were ~200KB/s, versus 260–525MB/s during the genuine swap-thrash incident. Adding RAM — the intuitive lever — would have changed nothing.

The structure was **cancel-all-then-repost-all**, which can never have a zero window: the book is genuinely empty between the DELETE and the first repost landing. Two earlier fixes (cancel moved later, cancel/repost parallelised) only *shrank* it.

What production venues do

Venue	Mechanism
Kalshi	Atomic `amend_v2` / `decrease_v2` — modify in place, never cancel-then-create — plus batched order operations, WebSocket, and FIX 4.4 for lowest latency.
Polymarket	Batch order placement (raised from 5 to 15 orders per call explicitly so makers can move a whole ladder in one round trip), batch cancel, and a WebSocket feed at ~100ms versus ~1s for REST polling.

Common thread: **amend atomically, batch the ladder, stay on a persistent connection, and react to market events rather than a timer.**

Fix 1 — make-before-break

An amend verb cannot be added to GameBull's matching engine (read-only). It does not need to be: **cancellation here is a direct MySQL DELETE we own outright**, not a matcher call. So the order was simply reversed — snapshot the resting row_ids, post the new ladder, then delete exactly the snapshotted rows. The book is never empty; worst case it is briefly *deeper* than intended, a strictly safer failure than empty.

The snapshot is essential: a fresh "cancel all my unmatched rows" at step 3 would also match the ladder just posted and wipe it.

Fix 2 — event-driven quoting

With the gap gone, requote frequency became free — so the loop was tightened 2000ms → 400ms (the 2s cadence had existed *only* because each cycle emptied the book). But measurement showed the loop was never the real constraint:

Measured	Result
Loop cadence	median **401ms**, p90 507ms, p99 739ms — healthy
Spot age in Redis when read	**27ms – 1,081ms**

The real constraint was **price staleness**, from two serial lags: `oracle-feed` throttles its Redis write to 250ms, and the quoter then polled that key every 400ms. Quoting off a quarter-second-old price during a fast move is exactly how a maker gets picked off — and it was the true cause of "quoting didn't keep up" during volatility.

Now: `oracle-feed` publishes every tick unthrottled on a pub/sub channel (the throttled write stays for pollers that don't need tick latency); the quoter subscribes on a dedicated connection and requotes immediately on a material move, with the interval retained as a **heartbeat** so τ-decay, gate flips and new markets are still picked up when spot is flat.

> **The half that is easy to miss:** triggering quickly is worthless if the quote then re-reads the stale key — which it did. A live-tick cache now supplies the price used for quoting, falling back to the polled key whenever the tick stream is stale or absent, so it can only improve freshness and never break quoting. Verified live: the quoter priced off $64,054 while the polled key still read $64,056.01 at 414ms of age.

Measured result

Configuration	Gaps/min	Mean	Max	≥3s
A — original cancel-then-repost	15.9	567ms	5,606ms	4
B — materiality gate only	6.5	448ms	1,203ms	0
C — make-before-break + event-driven	**0.6**	402ms	402ms	**0**

96% reduction, and the only surviving event occurs at τ≈300s — market *open*, i.e. the ~400ms before the first quote lands on a brand-new market. That is cold start, a different and separately fixable problem, not the cancel hole, which is structurally gone. Reaction latency improved in parallel: sub-300ms reactions 11% → 17%, p10 297ms → 215ms, fastest 28ms → 2ms. The median stays near 400ms because in *calm* periods the heartbeat correctly dominates — the gains concentrate exactly where they matter, in fast markets.

10.4 Adversarial QA on the hedging service

A deliberately hostile test suite found **three real production risks** that would only surface under load:

Severity	Defect	Fix
CRITICAL	Inventory adapter used Redis `KEYS` on **every poll** — blocks a shared production Redis.	Read the maintained active-markets SET via `SMEMBERS`.
HIGH	DynamoDB `Scan` was not paginated — past the 1MB cap it would **silently under-hedge**.	Loop on `LastEvaluatedKey`.
MEDIUM	NaN / Infinity propagation through the digital math when spot was zero or malformed.	Input guards and `safeNum`.

What the suite does **not** cover was documented explicitly rather than claiming full coverage.

10.5 A live defect found while preparing this document

The same pagination bug, in the application layer.

A verification trade filled **5/5 contracts** and the hedger correctly picked up the exposure — but the position **did not appear in the user's portfolio**.

Root cause: `portfolio()`, `settle()` and `roundReset()` each issued a bare `ScanCommand` on `bb_pending_bids`. DynamoDB caps every Scan response at **1MB** and returns a `LastEvaluatedKey` for the rest. The table had grown to **3,963 rows — only 1,199 fit on page one**. Everything beyond page one was invisible, silently and without error.

Impact:

- `portfolio()` — real positions vanish (wallet already debited, hedge already placed).
- `settle()` — **winners are credited from this same scan**. A winning position past page one **would never have been paid**.
- `roundReset()` — only the first page of positions was cleared.

Fix: a shared paginated `scanAll()` helper; all five call sites migrated. **Verified:** a fresh trade now fills 8/8 and the position appears immediately; a settled win reconciled exactly (–258 points staked, +500 paid, net **+242 = +$2.42**).

Note: this is the *identical defect class* already fixed in the hedging service (§10.4, HIGH). It was fixed there and **missed in the app** — a reminder that a bug class, once found, should be swept for across the whole codebase rather than patched at the site where it was noticed.

10.6 Working practices

- **Every risky change ships behind an environment-variable toggle** defaulting to the known-safe state, with a tested rollback script committed alongside.
- **Validate against live data rather than asserting from theory** — including pulling real Binance klines to prove a suspicious price move was genuine market movement and not a feed bug.
- **Self-correction is recorded, not hidden.** The hedge sign was gotten wrong twice and corrected by re-reading the actual order-book code rather than trusting memory; the CPU-versus-RAM misdiagnosis is documented above.
- **Contaminated data is reported, not presented.** When test traffic polluted an inventory-sizing analysis, that was stated rather than publishing a false-precision number.
- **Advice is corrected when later work invalidates it.** Raising the hedge notional cap was recommended as the obvious fix for §8.3, before the fee analysis of §8.7 showed that doing so alone would walk into a $429/window cost against $54 of revenue. The earlier recommendation is marked as incomplete in place rather than quietly replaced.

10.7 Market-rotation latency — a UX defect with two independent causes

Reported symptom: after a market ended, the replacement did not appear without a manual page refresh. Instrumented across a real rotation by polling `/api/markets` every 300 ms.

Phase	What was happening	Duration
Market expires	Still listed but flagged `expired`, so the UI filters it out and shows "Opening the next market…". `lifecycle()` ran on a blind 8-second poll and had not yet noticed.	**7.5 s**
Settlement prunes it	Removed from `predictor_active_markets`, but the replacement waited on the *entire* payout chain — distribution-engine round trips plus two hardcoded 2-second sleeps — because `settle()` ran inline.	**6.2 s**
Total blank-page window		**13.7 s**

The front end was **not** at fault — it polls at 1 s and did recover unaided. Fourteen seconds of an empty list simply reads as a frozen page, so users refreshed.

Two fixes, both in `app/server.mjs`:

- **`settle()` split into `settlePrune()` and `settlePayout()`.** Only the prune gates the market list; payout affects wallets and portfolio, which are polled separately and were already seconds behind. Payout now runs backgrounded, with errors logged rather than thrown — an unhandled rejection there would kill the lifecycle loop and stop rotation entirely.
- **Expiry-driven scheduling.** Expiry times are known exactly, so a timer now fires 300 ms after the known expiry instead of discovering it up to 8 s late. Re-armed in a `finally` block so a thrown lifecycle still reschedules; the 8-second interval is retained purely as a backstop against missed timers (clock jump, machine sleep).

Measured result: 13.7 s → 1.1 s, with the intermediate "pruned but no replacement" state eliminated entirely. Verified that the replacement market is quoted immediately (bid 63.7 / ask 66.7) and that the backgrounded payout still settles and accrues house P&L — the event log shows the new market opening a full 5 seconds *before* settlement completes, which is the intended ordering.

Known residual, stated rather than hidden. Payout is now fire-and-forget, so if the process dies within the ~5 s window after a rotation, that market's wallet credits are lost. This was already true — killing the app mid-`settle()` had the same effect — so it is not a regression, but it is now off the critical path and therefore less visible. A durable fix means a resumable settlement queue, which is real work rather than a tweak.

11 · Current System Status

Live-verified 31 July 2026, immediately prior to publication. Sections 8.5–8.7 and 10.7 reflect work completed the same day.

Component	Status	Evidence
BitBull app (:5050)	HEALTHY	Markets, Portfolio, Hedge Desk, Liquidity Watch all rendering live data
Real matching engine (:7001)	HEALTHY	Test order filled 8/8 at 46.9¢
Distribution engine (:3010)	HEALTHY	Settlement paid `winningAmount 500`; P&L reconciled to the point
Hedging sidecar (:8790)	HEALTHY	tick 14,844; 16 hedge fills to date. Sign verified live under load: delta −0.0889 → SHORT 0.0463 BTC
Persistent quoter	HEALTHY	Two-sided ladder reposting on ~2s cadence
Liquidity watcher	HEALTHY	Gap rate **15.9/min** → **0.6/min** after make-before-break; only survivor is market-open cold start (§10.3)
Docker stack (5 containers)	HEALTHY	Redis · MySQL · DynamoDB · SQS · Postgres, 9h uptime
Kronos-small fine-tune	COMPLETE	24.7M params, 33.8 min on T4. Beats mini on every measure; blend now best overall (§9.1)
Cross-market hedging	NO-GO	Awaiting long-horizon organic data re-run
Quoting latency	FIXED	Make-before-break + event-driven on tick pub/sub; loop 2s → 400ms (§10.3)
Directional lockout	NARROWED	Now τ≤30s only, was firing from τ=213s; both sides quote mid-market (§7.2)
Fair-value calibration	VERIFIED	+0.0pp at extremes (n=15,053); mid-range −9.4pp miscalibration found (§4)
Portfolio marking	FIXED	Mark-to-mid, was mark-to-model; mark-to-bid queued for when selling ships (§3.2)
Kalshi 15m watcher	COLLECTING	Live book + spot every 5s; spread benchmark in (§12.5), flow-impact pending
Price impact of user trades	TOO WEAK	Max inventory moves mid only 0.35¢; σ=0.10 suppresses it 100× (§7.4)
τ-scaling exponent	OPEN	√τ does not collapse the curve; α≈0.88–1.10, unstable — not shipped (§4.4)
Hedge δ curve	FIXED	Sized off the empirical curve, not Black–Scholes; was over-hedging 1.6–2.2× (3.15× ATM) (§8.5)
Hedge Desk skew display	FIXED	Now net contracts Σ(qYes−qNo) plus δ separately; was showing $66.8M on ~$1,000 of flow (§8.5)
Market-rotation latency	FIXED	Blank-page window **13.7s** → **1.1s** via expiry-driven scheduling + backgrounded payout (§10.7)
Hedge fee economics	CRITICAL	$429/window in taker fees vs $54 of spread revenue; 24¢ break-even spread against 3–6¢ observed (§8.7)
Hedge budget vs gamma	OPEN	$16M notional vs $10k budget — capital decision, now coupled to the fee wall (§8.7)

11.1 Recommended next steps, in priority order

1. **Fix the hedge economics before the hedge budget (§8.7).** Move hedge execution from taker to maker orders (an immediate halving of cost) and hedge a *fraction* — ~0.30 captures two-thirds of the available risk reduction at under a third of the cost. Both are free; neither needs a capital decision. **This supersedes the earlier framing of §8.3 as the top priority:** raising the notional cap without first fixing execution cost would convert a dormant problem into a live one.
2. **Then resolve the hedge-budget mismatch (§8.3, §8.7).** Still a capital allocation decision, but it must be taken jointly with the fee analysis — note that $536k of perp notional is *margin*, not capital: ~$72k at 10× leverage, ~$36k at 20×.
3. **Enable limit orders** to unlock genuine fee revenue (§8.4) — the order-entry path already exists.
4. **Run the cross-market ladder for days, not minutes**, and re-run the existing analysis (§9.2). Zero new code.
5. **Model the call-spread overlay** against the recorded ledger (§9.3). Analysis-only.
6. **Collect multi-day 1s BTC data, then fit the τ exponent, then re-fit the breakpoints — in that order** (§4.4). The mid-range miscalibration is real, but re-fitting the table alone is invalid because $\sqrt{\tau}$ does not collapse the curve; doing it first would encode a τ-average as if it were τ-independent.
7. **Re-validate the empirical curve** across multiple regimes and extend real breakpoints past 0.15% (§4.3).
8. **Sweep for the pagination defect class** across every remaining full-table scan (§10.5).

12 · How Polymarket and Kalshi Actually Make Money

Researched July 2026. Both venues changed their fee models materially during 2026, so figures below are current-as-of-writing and should be re-checked before being acted on. Sources listed in §12.8.

BitBull's central economic problem (§5, §8.3) is that **the house is the counterparty to every trade** and therefore absorbs all inventory risk. The two largest prediction-market venues in the world have both solved that problem — and neither solved it by hedging better. They solved it **structurally**, by never taking the position in the first place. This section explains exactly how, and which parts are transferable.

12.1 The core structural difference

> **BitBull today:** house quotes both sides via LMSR → house holds inventory → house needs a hedge → hedge cannot cover terminal gamma (§8.1) → residual risk paid for with spread.
>
> **Polymarket / Kalshi:** the venue **does not quote**. It runs a central limit order book where users trade with *each other*. The venue's revenue is a **fee on volume**, and its inventory risk is **structurally zero**.

Everything below follows from that one difference. Note this is a genuine trade-off, not a free win — see §12.6.

12.2 Polymarket: the full mechanism

Polymarket runs on Polygon with USDC as collateral, using Gnosis's **Conditional Tokens Framework** (CTF). The critical idea is the **complete set**.

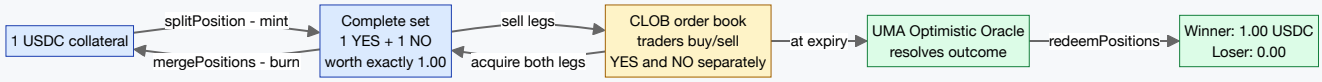
```
1 USDC  ⇄  1 YES token + 1 NO token
```

Because exactly one of the two outcomes pays $1 at resolution, a complete set is worth **exactly $1.00 by construction**. That identity is enforced by two smart-contract operations available to anyone at any time:

- `splitPosition` **(mint).** Deposit 1 USDC, receive 1 YES + 1 NO. The legs can then be sold independently.
- `mergePositions` **(merge).** Burn 1 YES + 1 NO, receive 1 USDC back — instantly, at par, **with no counterparty and no spread**.
- `redeemPositions` **(redeem).** After resolution, the winning token redeems for $1.00; the losing token for $0.00.

The three participant roles

Role	What they do	Why they do it / how they earn
Minter	Locks 1 USDC as collateral and mints a complete set — 1 YES + 1 NO.	Supplies the market's inventory. Typically a market maker who then sells one or both legs into the book. Bears no directional risk while holding the full set.
Trader	Buys or sells individual YES or NO tokens on the CLOB.	Directional opinion. Pays the taker fee; earns or loses on the outcome.
Redeemer	After resolution, redeems winning tokens for $1.00 each.	Realises the payout. Alternatively, a holder of both legs can **merge** back to collateral *before* resolution, exiting instantly at par.



> **Why the merge operation matters enormously for us.** It is a **counterparty-free, spread-free, slippage-free exit**. A Polymarket market maker holding an unbalanced book can buy the missing leg and merge back to cash at par. BitBull's house has **no equivalent**: to flatten inventory it must either find a user willing to take the other side, or hedge externally on a perp — which is precisely where §8.1 proves it structurally cannot fully succeed.

Resolution — UMA's Optimistic Oracle

Polymarket does not decide outcomes itself. A separate adapter contract routes resolution to **UMA's Optimistic Oracle v2**, whose design philosophy is that **a submitted answer is assumed true unless someone pays to challenge it**:

1. At expiry, the market contract requests the outcome, citing the resolution criteria fixed at market creation.
2. A **proposer** submits an answer and posts a **$750 USDC bond**.
3. A **2-hour challenge window** opens. If nobody disputes, the answer stands and the market settles.
4. A **disputer** may challenge by posting a matching bond. The dispute escalates to UMA's **DVM** (Data Verification Mechanism) — a token-weighted vote of UMA holders using a commit–reveal scheme, roughly 48h to commit and a further 24h to reveal.
5. The winning side **recovers its own bond and takes the loser's**. The loser forfeits theirs entirely.

The economics are the point: honest proposals are cheap, dishonest ones are expensive, and the bond makes challenging profitable — so policing is crowdsourced rather than administered.

> **Relevance to short-dated crypto binaries is limited.** A 2-hour challenge window plus a multi-day DVM escalation path is fundamentally incompatible with a **5-minute** market that must settle instantly. For a BTC price binary, resolution is a deterministic price lookup against an agreed feed — no human judgement, no dispute surface. BitBull's oracle problem is genuinely easier than Polymarket's, and UMA-style machinery would be over-engineering here. It is the *right* answer for subjective questions, not for ours.

12.3 The fee models — and a striking convergence

Kalshi

A CFTC-regulated designated contract market, so revenue is straightforwardly a trading fee:

$$\text{fee} = \lceil\ 0.07 \times C \times P \times (1 - P)\ \rceil$$

where C = contracts and P = price in dollars. The fee **peaks at P = $0.50** (where P(1−P) = 0.25, giving a maximum of **1.75¢ per contract**) and falls toward zero as price approaches 1¢ or 99¢. Maker fees are **25% of the taker fee**, and are **0% on most standard markets** — rising only for premium events. A July 2026 revision retained the 0.07 coefficient and introduced a per-category multiplier.

Polymarket

Historically fee-free; introduced taker fees on **30 March 2026**, expanding across categories. Structure as of July 2026:

Category	Max taker fee per 100 shares
Politics, finance, tech	$1.00
Sports, economics, culture, weather	$1.25
Crypto	**$1.75 — the highest of any category**
Geopolitical	$0.00 (fee-free)

Makers pay **nothing**. Instead, Polymarket runs a **Maker Rebates Program**: 15–25% of collected taker fees are redistributed **daily** to market makers in proportion to their share of executed maker liquidity in that specific market — 20% for crypto, 15% for sports, 25% for most others. In July 2026 the crypto rate eased slightly from 0.072 to 0.07.

> **The convergence worth noticing.** Two entirely independent venues — one a US-regulated exchange, one an on-chain protocol — have arrived at the **same fee shape**: proportional to **P·(1–P)**, peaking at 50/50, and at almost the same coefficient (**0.07**) and cap (**1.75¢/contract**) for crypto.
>
> That is not a coincidence. **P(1–P) is the variance of a Bernoulli outcome.** Both venues are charging in proportion to **how uncertain the market is** — which is also precisely where a market maker bears the most adverse-selection risk. Compare our own pin-risk widening term from §6.2:
>
> $$\text{pinRisk} = \text{gammaWiden} \cdot p(1-p) / \sqrt{\tau}$$
>
> We independently derived the **same p(1–p) core** for spread widening that they use for fee-setting. Their version is flat in time; **ours additionally scales by 1/√τ**, which is the correct refinement for short-dated binaries where risk concentrates into expiry.

12.4 How market makers actually earn on these venues

Revenue source	Mechanism
Spread capture	The classic source: quote both sides, earn the bid–ask on balanced two-way flow. Same economics as BitBull's vig (§5.2).
Maker rebates	Polymarket pays makers a daily share of taker fees; Kalshi charges makers a quarter of the taker rate or nothing at all. On both venues, **providing liquidity is subsidised rather than taxed**.
Complete-set arbitrage	If YES + NO trade below $1.00, buy both and `merge` for a risk-free profit. If above $1.00, `split` and sell both. This arbitrage is what **enforces** the $1 identity and is available to anyone with collateral.
Cross-venue arbitrage	The same BTC event is often priced on Polymarket, Kalshi, Deribit options and perpetual futures simultaneously. Discrepancies are tradeable, and a binary can be replicated with a tight call spread (§9.3) — so options markets bound what a binary *should* cost.
Favorite–longshot bias	Documented on Kalshi: contracts priced ≤10¢ lose more than 60% on average, while favourites are slightly underpriced. A maker who systematically leans against longshot buying captures that edge — and it is the same bias our simulated agent population models (§5.1).
Delta hedging	For crypto markets specifically, makers hedge directional exposure on perps — exactly the machinery of §8.2, with exactly the same terminal-gamma limitation of §8.1.

Why crypto is the most expensive category on both venues is itself informative. Short-dated crypto markets attract the fastest, best-informed flow — the underlying trades 24/7 on deep external venues, so any mispricing is arbitraged within seconds. Higher fees compensate makers for heavier adverse selection. Our own finding that short-gamma pin risk dominates the loss decomposition (§6) is the same phenomenon, observed from the other side of the trade. It also establishes **willingness to pay**: crypto traders demonstrably accept ~1.75¢ per contract.

12.5 A live benchmark — Kalshi's own 15-minute BTC market

Kalshi runs a series, **KXBTC15M** — "*BTC price up in next 15 mins?*" — which is the closest public analogue to BitBull's product: short-dated, same underlying, resolved off an external index (CF Benchmarks BRTI, 60-second average), but made by **many competing market makers** rather than a single house. A watcher (`research/kalshi_15m_watcher.py`) records its book alongside BTC spot every 5 seconds via Kalshi's public, unauthenticated market-data API.

Spread — the headline comparison

YES mid band	n	Kalshi median spread
0.00 – 0.05	12	**0.10¢**
0.05 – 0.15	11	**0.10¢**
0.15 – 0.35	25	1.00¢
0.35 – 0.65	27	1.00¢
BitBull, for comparison		**3¢ typical, 6¢ observed**

> Kalshi quotes a **flat 1.00¢** through the middle of the distribution and tightens to **0.10¢** in the tails — the opposite shape to ours, since our pin-risk term widens as certainty rises. We are **3–6× wider** at the money. This is not directly a criticism: they have many makers competing the spread down and diversified across thousands of concurrent markets, while BitBull is one house on one market with a $10k hedge budget (§8.3). But it is the number a user comparing venues would see first, and it bounds how much vig this product can realistically charge.

Kalshi also uses **tapered tick sizes** (`price_level_structure: tapered_deci_cent` — 0.1¢ steps in the 0–10¢ band, coarser in the middle) rather than our flat 0.1¢ everywhere. That is what lets a 0.10¢ spread exist at all in the tails, and is a cheap structural improvement available to us (§7.3 already established which increments are safe against the matching engine's float truncation).

What it is being collected for

The open question the watcher exists to answer is **§7.4**: how much *should* a user trade move our price? Kalshi's tape allows each price change to be decomposed into the part explained by BTC spot moving (information) and the residual correlated with traded volume (order-flow impact). That residual, expressed in cents per contract traded, converts directly into a σ or γ setting for our reservation price.

> **Not yet answered.** Early samples show price moving with volume while spot was flat, which would imply meaningful flow impact — but on a handful of observations, in a window where BTC barely moved, that is not a result. Reporting it as one would be exactly the false precision this document tries to avoid. The watcher needs hours spanning genuinely moving markets. **Collection limitation worth recording:** the API host is reachable only ~60% of the time from this machine, so the series has gaps by construction; every poll retries and failures are logged rather than silently dropped.

12.6 What BitBull can take from this

Idea	Problem it addresses	Assessment
Complete sets (split / merge)	§8.3 — house holds unbounded gamma with a $10k hedge budget against ~$16M worst-case notional	**Highest value.** Gives a counterparty-free, at-par inventory exit that no amount of perp hedging can replicate. Directly attacks the problem §8.1 proves is otherwise unsolvable. Needs no blockchain — it is an accounting construct: escrow $1, issue both legs, allow merge-back.
Taker fee ∝ p(1−p)	§8.4 — fee revenue is structurally $0	**High value, low effort.** Two independent venues validate both the shape and the price point. We could go further and charge **p(1−p)/√τ**, matching our own risk term — a defensible refinement for 5-minute markets, since neither venue's flat-in-τ fee reflects how sharply risk concentrates into expiry.
Maker rebates	§8.3 — house is the *sole* liquidity provider and bears 100% of inventory risk	**High value, medium effort.** Paying external makers a share of taker fees recruits others to absorb inventory. Changes the house from sole risk-bearer to venue operator. Requires user-to-user matching first.
User-to-user CLOB	§8.4 — every fill is house-vs-user, so there is nothing to charge a fee on	**Prerequisite for the two above.** The limit-order entry path already exists in the app; what is missing is enabling users to match against *each other* rather than only against the house ladder.
UMA-style optimistic oracle	Resolution integrity	**Not applicable.** Deliberately rejected: a 2-hour challenge window cannot serve a 5-minute market, and a BTC price binary resolves by deterministic feed lookup with no dispute surface.

12.7 The honest counter-argument

> **Moving to a pure CLOB is not a free win, and this section should not be read as recommending it wholesale.**
>
> - **Cold start.** A CLOB with no makers has **no liquidity and no price**. LMSR's defining property is that a quote *always* exists, at any size, from the first second of a market's life. That is precisely why a house-maker model was chosen — it solves bootstrapping, which a CLOB does not.
> - **The subsidy buys something real.** The b·ln2 loss (§5.2) is the price of guaranteed, immediate, two-sided liquidity. Removing the house maker removes that cost — and that guarantee with it.
> - **Revenue changes character.** A venue earns thin fees on *large* volume; a house maker earns a thick spread on *small* volume. Switching only pays if volume materialises, and Polymarket ran **fee-free for years** to build the volume that now makes fees viable.
> - **Regulatory divergence.** Kalshi's model presumes a regulated exchange; Polymarket's presumes on-chain settlement with an external oracle. Neither maps cleanly onto GameBull's existing platform without significant structural change.

The pragmatic reading: the highest-value items — **complete sets** and a **p(1−p)-shaped taker fee** — are adoptable *without* abandoning the house-maker model. Complete sets can coexist with LMSR quoting: the house keeps providing liquidity, but gains an at-par unwind path it currently lacks. That is an additive change to

inventory management, not a rewrite of the business model.

12.8 Sources

Kalshi fee formula and maker/taker structure — pm.wiki, PredictReport, Maker/Taker Math on Kalshi.
Polymarket fee structure and Maker Rebates Program — Polymarket Documentation, Polymarket Help Center, Phemex, Prediction Hunt.
Conditional Tokens split / merge / redeem — Polymarket CTF Documentation, Redeeming vs Splitting vs Merging.
UMA Optimistic Oracle resolution, bonds and DVM escalation — Polymarket UMA CTF Adapter, Polymarket Resolution Docs, TierZero.

13 · Appendix

13.1 Repository map

```
~/Desktop/gb-crypto-local/          Local exchange + BitBull app
  app/server.mjs                    API, trading, settlement, P&L accounting
  app/public/index.html             Front end (4 tabs)
  drivers/lib/pricing.mjs           Empirical curve + BS digital  ← §4
  drivers/lib/quoting.mjs           Stoikov + gamma widening      ← §6
  drivers/lib/risk.mjs              Inventory limits, quote sizing
  drivers/lib/{flow,matching,fees}.mjs
  drivers/lib/EMPIRICAL_VALIDATION.md   Methodology + results     ← §4
  drivers/mmp-pricing/              Persistent quoting worker
  drivers/market-generator/         Rolling markets (TENOR_MIN)
  drivers/sqs-bridge/               SQS → matching engine
  drivers/inventory-mirror/         House inventory → Redis
  drivers/liquidity-watcher/        400ms gap detector            ← §10.2
  docs/gamma-hedging-plan.md        Cross-market + options plan   ← §9
  data/backups/                     Timestamped rollback scripts

~/gb-crypto-hedging-service/        Read-only perp hedging sidecar ← §8.2
  src/core/digital.ts               Digital delta + gamma
  src/core/{gate,hedger,ledger,taper,exposure-recorder}.ts
  src/inventory/gamebull.ts         Redis inventory adapter
  src/venue/{binance-demo,dry-run}.ts
  src/http/server.ts                /health /state /metrics /config /kill
  scripts/phase0-analysis.ts        Cross-market feasibility       ← §9.2

~/gb-crypto-kronos/                 ML research                    ← §9.1
  finetune/calibration_report*.py   Calibration harnesses
  kaggle-package/                   Cloud GPU package
```

13.2 Running the stack

```
docker compose up -d                    # Redis, MySQL, DynamoDB, SQS, Postgres
node setup/create-schema.mjs            # idempotent schema bootstrap
node app/server.mjs                     # :5050 — also spawns persistent quoter
node drivers/liquidity-watcher/index.mjs # optional diagnostics
cd ~/gb-crypto-hedging-service && npm start   # :8790 (DRY_RUN default)
```

13.3 Glossary

Binary / digital option	Pays $1 if a condition holds at expiry, else $0.
Delta (∂p/∂S)	Sensitivity of value to spot. Diverges as τ→0 for a digital.
Gamma (∂²p/∂S²)	Rate of change of delta. The O(1/τ) term driving pin risk.
Pin risk	Risk of spot sitting at the strike near expiry, where value is maximally unstable.
LMSR	Logarithmic Market Scoring Rule — inventory-priced AMM. Bounded loss b·ln2.
b (liquidity param)	LMSR depth. GameBull names it `volatility`; default 500.
Avellaneda–Stoikov	Market-making model: shift reservation price by inventory, size spread by risk aversion k.
Adverse selection	Loss from trading against better-informed flow. Unhedgeable; only the vig pays for it.
Vig	The market maker's spread — its only revenue source.
MAE	Mean absolute error. Linear in error size — the right lens for pricing.

Brier score	Mean *squared* error. Punishes confident-and-wrong — the right lens for risk sizing.
τ (tau)	Time to expiry.
DRY_RUN	Hedger computes and logs orders without sending them. Default on.

> **Closing note.** The most valuable outputs of this work are arguably the **negative** results: that perps cannot hedge terminal gamma (§8.1), that the hedging budget is undersized by three orders of magnitude relative to worst-case gamma (§8.3), and that cross-market hedging did not clear its pre-registered thresholds (§9.2). Each of these closes off a direction that would otherwise have consumed real engineering time, and each is stated with the evidence and the sample-size caveats needed to re-open it properly.